CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

CAST: Carbon-Aware Spatio-Temporal Scheduling under Capacity Constraints

초록 리전 용량 안에서 작업을 깨끗한 시각과 리전으로 옮기는 탄소 인지 스케줄링에서, 남의 확정 계획을 보지 않고 미래 시작 시각을 정하면 시작 때 확인하지 않는 한 채점 작업의 9.0~17.0%가 상한을 넘는다. 제안하는 CAST는 매 슬롯 대기 작업 전부의 리전과 시각을 선형 계획으로 다시 풀어 지금 시작만 확정한다. 8개 리전 1년 시뮬레이션에서 CAST는 위반 없이 평균 배출계수로 센 실행 전력 배출을 무릎점에서 64.44~71.93%, 탄소만 볼 때 95.59~95.87% 줄였다. 위반이 없는 가장 가까운 두 설계보다 합성 워크로드에서 같은 고정 가중치로는 0.68~1.64%포인트 덜 배출했고, 무릎점 규칙으로는 같은 만큼 배출하며 31% 덜 기다리게 했다. Azure 트레이스에서는 예약을 새 도착 몫에 맞추면 CAST가 앞서고, 크게 잡으면 2.10%포인트 뒤졌다. CAST 무릎점보다 많이 줄이는 선행 정책 셋은 가장 붐빈 리전에 기준 피크의 2.2~9.9배 용량을 요구했다.

주제어: 탄소 인지 스케줄링, 워크로드 이동, 리전 용량, 선형 계획법, 롤링 지평

Abstract In carbon-aware scheduling that moves jobs to clean hours and regions within regional capacity, fixing future start times without seeing other jobs' committed plans lets 9.0–17.0% of scored jobs start over the ceiling unless capacity is checked at start time. We propose CAST, which at every slot re-solves the region and start of all waiting jobs by linear programming and commits only the jobs that start now. In a one-year simulation over eight regions, CAST has no violations and cuts operational emissions, counted with average emission factors, by 64.44–71.93% at the knee and by 95.59–95.87% when only carbon is weighed. Against the two closest designs that also have no violations, on a synthetic workload it emits 0.68–1.64 percentage points less at the same fixed weight and latency, and with its knee rule it matches their emissions while making jobs wait 31% less. On an Azure trace CAST leads when the reservation matches new arrivals and trails by 2.10 points when it is set large. The three prior policies that save more than CAST's knee need 2.2–9.9 times the baseline peak in their busiest region.

Keywords: Carbon-aware scheduling, Workload shifting, Regional capacity, Linear programming, Rolling horizon

1. 서론

데이터센터가 쓰는 전력이 빠르게 늘고 있다. 국제에너지기구는 2024년 약 415TWh였던 전 세계 데이터센터 전력 소비가 2030년 945TWh에 이를 것으로 전망하며 AI를 가장 큰 요인으로 지목했다[1]. 에너지 사용의 투명성이 규제 대상이 된 이상[2] 배출을 줄이는 일은 운영상의 과제가 되었고, 이미 가동 중인 인프라에 신규 설비 없이 적용할 수 있는 수단이 필요하다. 본 논문은 그 수단의 하나인 탄소 인지 스케줄링에서, 작업이 같은 깨끗한 시각과 리전으로 몰려 용량을 넘기는 문제를 다룬다.

배출량을 정하는 것은 소비 전력량만이 아니다. 전력 1 kWh를 만들 때 나오는 탄소량, 곧 탄소집약도는 그 순간 전력망의 발전원 구성에 따라 달라진다. 본 연구의 여덟 리전에서도 2025년 1월 8일부터의 실측 연평균이 인도 543.2 gCO_2/kWh, 프랑스 13.6 gCO_2/kWh로 약 40배 벌어지고, 하루 안에서도 캘리포니아는 날마다 최댓값이 최솟값의 평균 3.93배다. 클라우드는 이미 여러 리전에 흩어져 있고 배치 작업은 완료 시점에 여유가 있으므로, 이 격차를 쓸 조건은 갖추어져 있다.

선행 연구는 이 격차를 시간 축이나 공간 축에서 써 왔다. 시간 축에서는 구글의 CICS[3]가, 공간 축에서는 CASPER[4]와 VMware의 전역 부하 분산[5]이 대표적이다. 두 축을 함께 다룬 Sukprasert 등의 상한 분석[6]은 공간 이동이 지배적이라고 결론지었으나 결합 분석에서는 리전 용량을 두지 않았다. 용량을 다룬 연구는 그 대신 다른 것을 양보한다. CarbonFlex는 단일 클러스터 안의 시간 이동에 한정되고[7], Caspian[8]은 마감을 목적함수의 벌점으로 완화한다. 리전별 상한과 하드 마감을 함께 둔 Zanotto 등[9]은 요청이 도착할 때 그 요청 하나의 자리만 정하고 다시 풀지 않으며, 상한을 어떻게 정할지는 향후 과제로 남겼다.

용량을 빼면 무슨 일이 생기는지는 본 연구의 자료에 그대로 드러난다. Sukprasert 등의 정책을 원문 수식대로(4.8절의 변경 제외) 다시 구현해 돌리면 작업 143,200건 가운데 142,841건, 곧 99.75%가 프랑스 한 리전으로 몰리고 배출은 96.63% 준다(4.8절). 그러나 이 배치가 위반 없이 돌려면 리전 하나가 동시에 129건을 돌릴 수 있어야 한다. 홈 리전에서 즉시 실행할 때 어느 리전도 13건을 넘긴 적이 없으므로 그 9.9배다. 가장 깨끗한 리전 하나로 전부 보내는 것이 최적해가 되기

때문이며, 계산 위에서만 성립하는 배치다. 용량을 제약으로 두는 순간 문제가 달라진다. 어느 리전이 깨끗한가가 아니라, 그 리전이 받아 줄 수 있는 만큼만 보내고 남은 작업을 어디에 언제 둘 것인가가 된다.

이 문제가 어려운 것은 가장 깨끗한 리전의 가장 깨끗한 시각이 모든 작업에게 같기 때문이다. 미래 시작 시각을 미리 확정하는 설계에서 각 작업이 제 예측 최저 시각을 고르고 용량은 이미 시작한 작업만 보고 확인하면, 아직 시작하지 않은 다른 작업의 확정 계획이 보이지 않아 작업들이 같은 골, 곧 하루 중 탄소집약도가 가장 낮은 구간으로 계속 몰린다. 위치와 작업 길이를 받아 가장 깨끗한 시작 창을 돌려줄 뿐 다른 작업의 계획이나 용량은 입력으로 받지 않는 Carbon Aware SDK[10] 같은 도구로 작업마다 시작 시각을 따로 고르면 이 꼴이 된다. 같은 선형 계획, 예측, 가중치 규칙 위에서 견주면, 다른 작업의 확정 계획을 보지 않고 확정하는 설계는 시작 때 확인하지 않으면 채점 작업의 9.0~17.0%가 상한을 넘긴 순간에 시작한다(4.4절). 시작 때 확인해 자리가 없으면 빈 리전으로 보내면 넘치지 않는다. 몰림 자체는 단일 HPC 시설의 시간 이동[11]과 실시간 수요반응[12]에서도 보고되었다. 용량을 두느냐만이 아니라 어떻게 두느냐가 결과를 가른다.

본 논문은 탄소 인지 스케줄링 시스템 CAST를 제안한다. CAST는 확정 계획을 들고 있지 않는다. 매 슬롯 대기 작업 전부의 리전과 시작 시각을 하나의 선형 계획으로 다시 풀고 지금 시작하는 작업만 확정하므로, 미래 슬롯을 미리 차지한 계획이 없고 위반도 없다(4.4절). 위반이 없는 가장 가까운 두 설계, 곧 다시 풀지 않는 설계와 시작 때 빈 리전으로 돌리는 설계와 견주면, 같은 고정 가중치로는 같은 지연에서 조금 덜 배출하고, 실제 운용하는 무릎점 규칙으로는 같은 만큼 배출하면서 작업을 덜 기다리게 한다(4.4절, 4.7절). 이를 위해 CAST는 세 가지를 함께 한다. 용량을 입장 때 거르는 조건이 아니라 대기 중인 모든 작업이 함께 보는 제약, 곧 공유 제약으로 두고, 매 슬롯 아직 시작하지 않은 작업 전부를 다시 풀며, 상한을 넘긴 양에 가장 큰 벌점을 둔다. 뒤의 둘이 요청마다 한 번 자리를 정하고 다시 풀지 않는 선행 연구[9]와 다른 점이다. 다시 푸는 롤링 지평 자체는 표준 기법이며 Caspian[8]도 매시간 다시 풀지만, Caspian은 용량을 지키려고 마감을 양보한다. 그 위에서 탄소와 지연의 가중치는 슬롯마다 고른다.

예측 단계가 여덟 리전의 향후 24시간 탄소집약도를 LSTM으로 내면, 배정 단계가 매 1시간 슬롯마다 아직 시작하지 않은 작업 전부의 실행 리전과 시작 슬롯을 하나의 선형 계획으로 함께 정한다. 지금 시작하는 작업만 확정하고 나머지는 다음 슬롯에 새 예측으로 다시 푼다. 배치는 예측값으로 결정하되 배출량은 실측값으로 세어 예측 오차의 대가가 결과에 남도록 했다.

본 논문을 이루는 기법 하나하나는 새롭지 않다. 이미 정한 배정을 함께 보는 용량 제약은 Zanotto 등[9]이, 매 슬롯 다시 푸는 롤링 지평은 Caspian[8]이 쓴다. 본 연구의 몫은 둘이다. 첫째, 확정 계획을 들고 있지 않고 매 슬롯 리전과 시각을 함께 다시 풀어 지금 시작만 확정하는 CAST를 제시하고, 위반이 없는 가장 가까운 두 설계와 견준다. 합성 워크로드에서 같은 지연·같은 고정 가중치로는 다시 풀지 않고 도착 때 확정하는 도착 고정형[9]보다 1.19~1.64%포인트(실행 시간 가중·작업당 지연 짝), 남의 계획을 보지 않고 확정하되 시작 때 빈 리전으로 보내는 대피형보다 0.68%포인트(신뢰구간 0.46~0.91) 덜 배출한다. 실제 운용하는 무릎점 규칙으로는 같은 지연의 도착 고정형과 배출이 같고(71.93% 대 72.08%, 차이 없음) 작업을 31% 덜 기다리게 한다(평균 1.47 대 2.13시간). Azure에서는 예약을 새로 도착하는 몫에 맞추면($\rho \le 6$) CAST가 앞서고, 넘어온 VM을 두 번 세어 커진 대표 설정($\rho=13$)에서는 도착 고정형이 2.10%포인트 앞선다. 이 이점은 대부분 매 슬롯 다시 풀며 지금 시작만 확정하는 데서 오며, 미래 슬롯 행은 0.12~0.72%포인트를 더하고 초과 벌점과 예약은 이 워크로드에서 몫이 없다(4.4절, 4.7절). 둘째, 선행 정책을 본 연구가 정한 상한이 아니라 각 정책이 위반 없이 돌기 위해 요구하는 필요 용량으로 다시 잰다. CAST의 무릎점보다 많이 줄이는 세 정책은 가장 붐빈 리전에 기준 피크의 2.2~9.9배 용량을 전제하며, 탄소만 볼 때 CAST는 상한 17에서 95.65%로 한 리전에 29칸을 요구하는 CASPER(95.30%)와 비슷하게 줄인다(회계에 따라 부호가 바뀌는 작은 차). 리전별 최댓값을 모두 더한 설비 총량으로는 CASPER(46칸)가 CAST 무릎점(117칸)보다 작으므로, 이는 리전마다 용량이 정해진 조건에서의 비교다(4.8절). 가중치를 슬롯마다 고르는 무릎점 규칙은 목적식과 같은 자의 같은 지연 고정 가중치보다 합성 워크로드의 아홉 조건 가운데 여덟에서 더 배출하므로 기여로 내세우지 않고 운용 규칙으로만 쓴다(4.5절). 평균 배출계수로 셀 때 상한이 주로 걸리는 리전(캘리포니아와 프랑스)과 시간 이동의 절감이 큰 리전(나라 단위 구역 자료에서 인도와 일본이며, 산정 방식이 바뀐 것으로 보이는 자료라 이 결론은 자료에 기댄다)도 함께 잰다(4.6절).

모든 작업이 지연 예산 안에서 모든 리전으로 갈 수 있는 조건으로 1년치 탄소집약도를 시뮬레이션한 결과 CAST는 홈 리전 즉시 실행 대비 배출을 무릎점에서 64.44~71.93%(평균 지연 39.2~47.8 ms), $\alpha=1$에서 95.59~95.87% 줄였다. 범위는 예측기를 24시간 전 값,

LSTM, 완전예지로 바꾼 폭이며, 본 연구의 LSTM으로는 71.93%(47.8 ms)와 95.65%(120.0 ms)다. 무릎점의 폭은 대부분 무릎점이 예측기마다 다른 α를 고른 데서 오며, LSTM에서 찾은 같은 지연의 고정 α 하나를 세 예측기에 쓰면 72.6~74.0%로 덜 흔들린다(4.5절). 어느 경우든 채점 구간의 작업 143,200건 전량이 마감 안에 처리되었고 상한을 넘긴 배정은 없었으며, 슬롯 안 도착을 미리 알지 않고 도착 순간에 배정해도 마찬가지였다. 위반과 필요 용량은 회계와 무관하고, 절감률은 평균 배출계수로 센 값이다.

2. 관련 연구

선행 연구는 탄소 인지 스케줄링이 배출을 얼마나 줄일 수 있는지를 여러 방향에서 보여 왔다. 시간 축만 쓰는 연구가 먼저 나왔다. Wiesner 등은 한 데이터센터 안에서 작업을 미루는 것만으로 얻는 절감을 네 지역의 전력 자료로 보였고 자원 제약은 두지 않았다고 밝혔다[13]. CarbonScaler는 한 리전 안에서 작업 하나의 서버 수를 탄소 예보에 맞춰 늘리고 줄이며[14], GAIA는 한 리전 클러스터에서 탄소, 완료 시간, 비용 사이의 맞바꿈을 다룬다[15]. GREEN은 기계학습 클러스터의 스케줄러 안에서 하루 총 용량을 지키며 작업을 저탄소 시각으로 옮긴다[16]. 넷 모두 작업이 어디서 도는지는 바꾸지 않는다. 리전과 시각을 함께 옮기고 리전 용량과 작업 마감을 함께 제약으로 넣고 보면 세 가지가 설계에서 걸린다.

첫째, 두 축을 결합한 선행 연구는 리전 용량과 작업 마감 가운데 하나를 양보한다. Sukprasert 등은 용량이 공간 이동의 이득을 좌우함을 스스로 확인하고도 결합 분석에서는 용량을 두지 않았고[6], Attenni 등도 자원이 항상 가용하다고 가정한다[17]. Lechowicz 등의 CarbonClipper는 작업 하나를 마감 안에서 리전 사이로 옮기는 온라인 문제에 경쟁비 보장을 주었으나, 여러 작업이 자원을 두고 다투는 상황과 데이터센터 용량은 정식화에 없다고 적었다[18]. Hall 등은 하루 전 계획과 실시간 배치로 부하를 시공간으로 옮기며 확률적 성능 보장을 주지만, 대상은 개별 작업이 아니라 집계 부하다[19]. Caspian은 반대로 용량을 하드 제약으로 두는 대신 마감을 목적함수의 벌점으로 완화하며[8], 본 연구 자료에서 재현하면 5,582건, 곧 채점 작업의 3.90%가 마감을 넘긴다(4.8절). Caspian은 매시간 다시 풀지만 용량을 지키려고 마감을 양보한다. 탄력적 확장으로 용량 가정을 방어할 수도 있으나, 본 연구는 특정 시각의 특정 리전에 그 여유가 늘 있다고 보지 않는다.

둘째, 용량을 두는 연구 가운데 여럿은 작업이 들어오는 순간에 한 번 판단하고 다시 보지 않는다. Zanotto 등은 본 연구와 가장 가깝다. 리전별 동시 실행 상한, 작업별 하드 마감, 자체 예측기를 갖춘 쿠버네티스 구조로, 요청마다 도착 시점에 기존 배정을 두고 정수 계획으로 리전과 시각을 정하며 그 뒤에는 다시 풀지 않는다. 다만 평가는 1개월 동안 1,000건 묶음을 반복한 것이고 비교 대상은 라운드로빈 하나이며, 상한을 어떻게 정할지는 향후 과제로 남겼다[9]. 이 설정은 4.4절의 대조군으로 재현한다. CICS는 하루 전에 산정한 자원 곡선 하나로 다음 날의 유연한 부하를 묶으므로 개별 작업의 마감을 보지 못한다[3]. CarbonFlex는 단일 클러스터 안에서 용량을 보지 않는 시간 이동이 저탄소 시각에 부하를 모은다는 점을 지적했으나, 리전 사이의 이동은 다루지 않는다[7]. COAST도 작업을 각자 저탄소 시각으로 보내면 같은 슬롯에 몰린다는 데서 출발해, 혼잡을 하드 용량 제약이 아닌 소프트 벌점으로 두고 작업들 사이의 혼잡 게임으로 시작 시각을 나눈다[11]. 대상은 단일 HPC 시설의 시간 이동이며, 예측이 좋아질수록 더 몰린다는 점은 다루지 않는다. 용량을 입장 조건으로만 두면 작업 하나가 들어올 때 그 작업이 고른 시각에 자리가 있는지만 묻고, 여러 작업이 같은 저탄소 시각을 노리고 있다는 사실은 그 시각이 찬 뒤에야 드러난다. 이미 정한 배정을 모두 보는 Zanotto 등의 방식은 이 몰림을 상한에서 멈추게 하지만, 한 번 정한 자리는 나중에 더 깨끗한 자리가 보여도 바꾸지 않는다.

셋째, 탄소와 그 밖의 목적 사이의 가중치가 고정 상수다. CASPER는 탄소와 서버 프로비저닝 비용 사이의 가중치를 고정하며 저자들 스스로 다른 값에서는 결과가 달라질 것이라고 밝혔다[4]. CICS 역시 탄소 비용과 피크 전력 비용의 가중합을 쓰고[3], 두 축을 하나의 정수 계획으로 결합한 Caspian도 가중치를 사람이 고르며 그 값을 신중하게 정해야 한다고 결론지었다[8]. 그 값을 미리 정할 근거는 주어지지 않는다.

본 연구는 이 세 가지를 다음과 같이 다룬다. 마감은 하드 제약(식 (4))으로 양보하지 않고, 용량은 초과량에 가장 큰 벌점을 두어(식 (3), (6)) 다른 길이 없을 때만 넘기며 실험에서는 넘긴 적이 없다. 용량은 입장 조건이 아니라 모든 대기 작업이 함께 보는 제약으로 두어 리전과 시작 시각을 매 슬롯 한 문제로 다시 푼다(3.4절). 가중치는 슬롯마다 이상점에 가장 가까운 후보로 정한다(3.4절). Zanotto 등과 비교하면 본 연구는 1년 동안 이어지는 작업 흐름, 탄소 인지 선행 정책 넷과의 같은 회계 비교, 상한 값을 바꿔 가며 본 분석을 더 갖추었고, 실제 클러스터 프로토타입은 갖추지 못했다. 본 연구가 재현한 선행 정책 가운데 리전 용량을 입장 조건으로 두는 것은 없다. CICS는 자기 자원 곡선으로, CASPER는

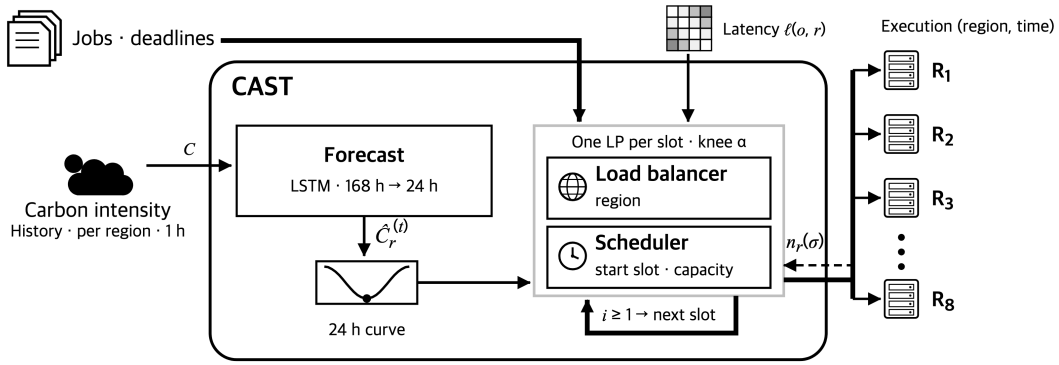


그림 1 CAST의 전체 파이프라인. 굵은 선은 작업, 가는 선은 탄소·지연 자료, 파선은 리전별 점유가 스케줄러로 다시 들어가는 길이며, 회색 테두리 안의 로드밸런서와 스케줄러는 매 슬롯 하나의 선형 계획으로 함께 풀린다.

Fig. 1 Overall pipeline of CAST. Thick lines carry jobs, thin lines carbon and latency data, and the dashed line returns each region's occupancy to the scheduler; the load balancer and scheduler inside the gray frame are solved together as one linear program per slot.

스스로 정한 서버 용량으로 묶을 뿐 리전 상한을 두지 않고, Sukprasert는 용량을 두지 않으며, Caspian은 용량을 하드 제약으로 두므로, 용량을 시작할 때만 보는 설계에서 생기는 몰림은 이들의 평가에서 드러날 자리가 없었다.

3. CAST 시스템

CAST는 예측기, 로드밸런서, 스케줄러의 세 모듈로 구성된다. 예측기는 리전별로 향후 24시간의 탄소집약도를 산출한다. 로드밸런서는 각 작업을 어느 리전에서 돌릴지를, 스케줄러는 그 리전에서 언제 시작할지를 정한다. 다만 로드밸런서와 스케줄러는 차례로 도는 두 단계가 아니라, 매 1시간 슬롯마다 하나의 선형 계획 안에서 함께 풀리는 두 축이다. 지금 시작하는 작업만 확정해 장부, 곧 리전마다 지금 도는 작업 수를 적어 두는 기록에 올리고, 나머지는 다음 슬롯에 새로 발행된 예측으로 다시 푼다. 그림 1이 세 모듈과 모듈 사이를 오가는 값을 보인다.

설계 원칙은 셋이다. 첫째, 용량은 입장 때 확인하는 조건이 아니라 대기 중인 모든 작업이 함께 보는 제약으로 두고, 리전과 시각도 그 제약 아래에서 한 문제로 정한다(3.4절). 둘째, 확정 계획을 들고 있지 않고 매 슬롯 다시 풀어 지금 시작하는 작업만 확정한다. 셋째, 배치 결정은 예측값으로, 배출량 산정은 실측값으로 한다. 둘 다 예측값을 쓰면 예측 오차의 대가가 결과에서 사라지기 때문이다.

세 모듈이 지향하는 문제를 먼저 1년치를 한 번에 보는 참조 문제로 적는다. 작업 집합을 J, 리전 집합을 R로 둔다. 슬롯 길이는 탄소집약도 자료의 해상도에 맞춘 1시간이다. 작업 j는 출발 리전 o_j, 제출 시각 s_j, 실행 시간 d_j, 지연 등급 k_j, 지연 예산 L_j를 가지며 마감은 $D_j = s_j + L_j + d_j$다. L_j는 시작을 미룰 수 있는 시간이자 왕복 지연의 상한으로 함께 쓰며, 둘은 시간 단위로 환산해 견준다. 지금 자료에서는 왕복 지연(최대 244 ms)이 가장 짧은 예산(1초)보다 짧아 둘을 따로 두어도 결과가 같아, 구현은 식 (5)의 검사를 따로 두지 않았다. 리전 r은 동시에 돌릴 수 있는 작업 수의 상한 cap_r을 가진다. 정하는 값은 실행 리전 $r(j)$와 시작 시각 τ_j 둘이고, 리전 r에서 시각 t에 돌고 있는 작업 수를 $n_r(t)$로 쓴다. t는 식 (1)(3)(6)에서 연속 시각이고, 3.3절부터는 슬롯 번호다.

목적은 사전식이다. 먼저 상한을 넘긴 양 Z를 최소화하고, 그 위에서 지연 Λ가 주어진 예산 $\mathcal{L}$를 넘지 않는 배정 가운데 배출 E를 최소화한다. 4장에서 고정 α를 같은 지연의 짝으로 견주는 평가가 이 꼴이다.

$$E = \sum_{j\in J} P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t)\,dt \tag{1}$$

$$\Lambda = \sum_{j\in J} d_j\, \ell(o_j, r(j)) \tag{2}$$

$$Z = \sum_{r\in R} \int \max\{0, n_r(t) - \mathrm{cap}_r\}\,dt \tag{3}$$

여기서 $C_r(t)$는 리전 r의 실측 탄소집약도, P는 작업의 소비 전력, $\ell(o, r)$은 출발 리전에서 실행 리전까지의 왕복 지연이다. E는 총 배출량, Λ는 리전 간 경로를 쓴 시간만큼 왕복 지연을 센 원격 실행 비용(이하 지연이라 부르며, 시작을 미룬 시간인 미룸과 다르다), Z는 상한을 넘긴 작업 수를 시간으로 적분한 초과량이다. 매 슬롯 푸는 식 (7)~(9)는 Λ 대신 가중치 α로 두 양을 맞바꾼다. 제약은 세 가지다.

$$s_j \le \tau_j, \quad \tau_j + d_j \le D_j \qquad \forall j \in J \tag{4}$$

$$\ell(o_j, r(j)) \le L_j \qquad \forall j \in J \tag{5}$$

$$n_r(t) \le \mathrm{cap}_r + z_r(t),\ z_r(t) \ge 0 \quad \forall r \in R,\ \forall t \tag{6}$$

식 (4)는 실행이 제출 이후에 시작해 마감 이내에 끝나야 한다는 뜻으로, 마감은 무를 수 없는 하드 제약이다. 식 (5)는 지연 예산을, 식 (6)은 용량을 정한다. 왕복 지연은 응답 경로의 지연으로만 세고 완료 시각에는 더하지 않는다. 상한을 넘긴 양 $z_i(t)$를 변수로 두고 그 적분이 식 (3)이 된다.

식 (6)이 본 연구의 형식화가 선행 연구와 갈라지는 지점이다.

용량을 두지 않으면 최적해는 가장 깨끗한 리전으로 몰린다. 용량을 무를 수 없는 제약으로 두면 마감이 코앞인 작업이 상한보다 많이 몰리는 순간 해가 없다. 초과를 변수로 두면 항상 해가 있고, 용량을 얼마나 어겼는지가 수식 밖의 사정이 아니라 목적함수의 값이 된다.

작업은 둘로 갈린다. 제출 슬롯 말고는 시작할 정수 슬롯이 없는 작업, 곧 $\lfloor D_j - d_j \rfloor \le \lfloor s_j \rfloor$인 작업을 미룰 수 없는 작업이라 부르고, 나머지를 미룰 수 있는 작업이라 부른다. 합성 워크로드에서 미룰 수 없는 작업은 건수로 59.1%지만 실행 시간이 초에서 분 단위라 총 작업시간의 1.3%에 그친다.

식 (1)~(6)을 1년치 전체에 대해 한 번에 풀지 않은 것은 시스템을 온라인으로 두기 위해서다. 한 번에 풀면 모든 작업과 1년치 탄소집약도를 미리 알아야 한다. 그래서 (1)~(6)은 참조 문제로 두고, 실제로는 매 슬롯 3.3절의 식 (9)~(12)를 푼다.

3.1 예측기

예측기는 리전마다 독립인 2층 LSTM이며, 은닉 크기는 64, 드롭아웃은 0.2이고 마지막 은닉 상태를 선형층으로 24개 값에 잇는다. 과거 168시간의 탄소집약도, 무탄소·재생 전력 비율, 시각, 요일, 월의 주기 변수, 공휴일 여부를 받아 향후 24시간을 한 번에 출력한다. 캘리포니아, 텍사스, 독일의 세 리전은 풍속, 일사량, 기온(Open-Meteo 과거 기상 API의 시간 단위 값)을 입력에 더하고, 향후 24시간의 같은 기상 변수(24×3=72개)도 출력층에 함께 넣는다.

학습은 2021~2023년, 검증은 2024년, 시험은 2025년으로 연도를 끊었고, 스케줄링 실험은 시험 연도 위에서만 돈다. 입력은 학습 연도에만 맞춘 최솟값–최댓값 척도로 0~1에 옮긴다. 손실은 정규화한 값의 평균 제곱 오차이고, Adam(학습률 0.001, 가중치 감쇠 0.0001)으로 배치 64씩 학습하며 기울기 노름을 1.0으로 자른다. 검증 오차가 8 학습 회차(epoch) 동안 줄지 않으면 학습률을 절반으로 낮추고, 15 회차 동안 줄지 않거나 가장 좋았던 회차 뒤 30 회차가 지나면 멈추되, 최소 20 회차, 최대 150 회차를 돈다. 모델은 원래 단위로 되돌린

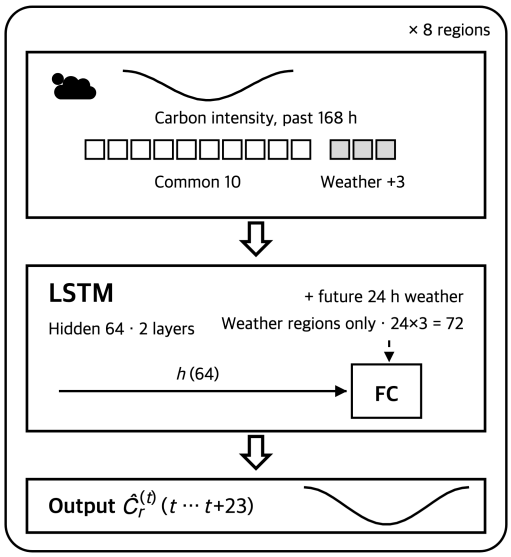


그림 2 리전별 LSTM 구조. 모델은 리전마다 독립이다.
Fig. 2 Per-region LSTM architecture. One model per region.

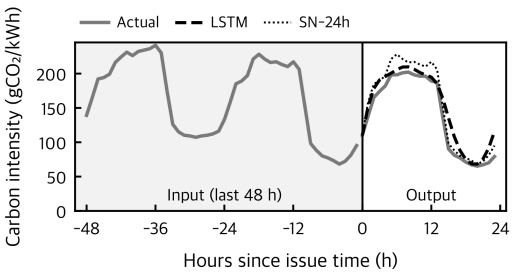


그림 3 캘리포니아의 탄소집약도 예측 예. 한 구간이며 왼쪽은 입력 168시간 중 마지막 48시간이다. 이 구간의 LSTM 평균 절대 오차는 12.2 gCO₂/kWh이고, 점선으로 겹친 24시간 전 값(SN-24h, 어제 같은 시각의 값) 예측은 14.0이다.
Fig. 3 Example carbon intensity forecast for California. One window; the left part shows the last 48 of the 168 input hours; the mean absolute error over this window is 12.2 gCO₂/kWh, against 14.0 for the dotted SN-24h forecast (the value at the same hour on the previous day).

검증 평균 절대 오차가 가장 작은 회차의 것을 쓰며, 난수 시드는 42다. 구조는 그림 2와 같고, 그림 3은 그 모델이 실제로 낸 예측 한 구간이다.

하루 앞 탄소집약도 예측은 선행 연구가 따로 다룬 문제이고[20] 본 연구의 기여가 아니다. 예측기가 뒤의 두 모듈에 넘기는 것은 값 하나하나보다 곡선의 모양, 곧

리전과 시각에 따라 탄소가 어디서 얼마나 깊이 내려가는지다. 로드밸런서와 스케줄러는 그 모양을 보고 자리를 고르기 때문이다. 예측 오차가 스케줄링 결과에 어떻게 이어지는지, 그리고 24시간 전 값을 그대로 쓰는 단순한 예측기와 비교한 결과는 4.2절에 있다.

여기까지가 예측기다. 이제부터는 그 예측을 받아 자리를 고르는 두 모듈을 적는다.

3.2 로드밸런서

로드밸런서는 작업마다 실행 리전을 고른다. 후보는 여덟 리전 전부이며, 식 (5)를 어기는 리전, 곧 출발 리전에서의 왕복 지연이 지연 예산보다 긴 리전은 후보에서 뺀다. 본 연구 자료에서는 지연 예산의 최솟값 1초가 왕복 지연의 최댓값 244 ms보다 커서 이 제약이 한 번도 걸리지 않지만, 예산이 더 짧은 워크로드에서는 이것이 공간 이동의 범위를 정한다. 왕복 지연은 전 구간에서 고정된 8×8 실측 행렬에서 읽는다(4.1절).

후보의 비용은 정규화한 탄소 비용과 지연 비용의 가중합에 실행 시간을 곱한 값이다. 후보는 리전 r과 i의 쌍이며, i는 지금부터 몇 슬롯 뒤에 시작하는지로 3.3절에서 다룬다.

$$c_j(r,i) = d_j\left[\alpha \tilde{C}_j(r,i) + (1-\alpha)\tilde{\ell}(o_j, r)\right] \quad (7)$$

$$\tilde{C}_j(r,i) = \frac{\hat{C}_r^{(t)}(t+i, d_j)}{\max_{r',t'} \hat{C}_{r'}^{(t)}(t')}, \qquad \tilde{\ell}(o,r) = \frac{\ell(o,r)}{244} \quad (8)$$

여기서 $\hat{C}_r^{(t)}(t+i, d_j)$는 슬롯 t에 발행된 예측에서 $t+i$부터 d_j 동안의 평균 탄소집약도이고, 분모는 그 슬롯에 발행된 예측의 리전·지평 전체 최댓값이다. 지금 슬롯($i=0$)의 첫 한 시간은 예측 대신 그 시각의 실측을 쓰고, 평균 창은 예측 지평 끝에서 잘라 그보다 긴 작업은 지평 안의 평균으로 판단하며, $i=24$는 $t+23$의 예측으로 본다. 지연 비용은 전 구간에서 고정된 값인 지연 행렬의 최댓값 244 ms로 나눈다. 탄소 비용만 슬롯별로 정규화하는 것은 슬롯 간 격차의 절대 크기를 가중치 해석에서 떼어내기 위함이며, 그래서 같은 α라도 슬롯마다 kg과 ms의 교환비가 다르다. 실행 시간 d_j를 곱한 것은 긴 작업이 그만큼 오래 배출하고 오래 원격 리전을 점유하기 때문이며, $\alpha=1$이면 식 (7)의 합이 예측 배출량에 비례한다.

용량을 보지 않는 선행 정책은 작업을 가장 깨끗한 리전으로 몰 수 있다(1장의 Sukprasert 공간+시간). CAST의 로드밸런서는 각 리전이 받아 줄 수 있는 만큼만 보낸다. 그 '만큼'은 3.3절의 상한 행이 정하고, 남은 작업은 다음으로 깨끗한 리전이나 다른 시각으로 간다. 1년 동안 실제로 만들어진 리전 간 흐름은 4.3절에서 본다.

3.3 스케줄러

스케줄러는 작업마다 시작 슬롯을 고르고, 리전별 장부로 용량을 지킨다. 매 슬롯 t마다 아직 시작하지 않은 작업을 모아 대기 집합 Q를 만든다. 미룰 수 있는 작업은 언제 도착했든 시작하기 전까지 Q에 남고, 이번 슬롯에 도착한 미룰 수 없는 작업도 Q에 들어간다. 슬롯 t에 제출되는 작업은 슬롯 시작 시점에 모두 알려진 것으로 보고 한 번에 배정한다. 최대 1시간을 미리 보는 것이며, 4장의 모든 대조군에 같게 적용한다.

$i=0$은 지금, 곧 $\max(t, s_j)$에 시작하는 것이고, $i\geq 1$은 $t+i$에 시작하는 것이다. i의 상한은 마감과 예측 지평 가운데 이른 쪽, 곧 $h_j = \lfloor \min(D_j - d_j,\ t+H) - t \rfloor$이며 지평 H는 24다. 미룰 수 없는 작업과, 여유가 한 슬롯보다 적게 남은 작업($(D_j - d_j) - t < 1$)은 $i=0$만 허용한다. 후보를 예측 지평 안으로 제한한 것은 예측이 24시간까지만 있어 그 너머는 비교할 근거가 없기 때문이다.

슬롯 t의 배정 문제는 다음과 같다. x_{jri}는 작업 j를 리전 r에서 i 슬롯 뒤에 시작하게 두는 변수다.

$$\min \sum_{j\in Q}\sum_{r,i} c_j(r,i)\,x_{jri} + M_A \sum_{r,\kappa} z^A_{r\kappa} + M_B \sum_{r,\sigma\geq t+1} z^B_{r\sigma} + \varepsilon \sum_{j\in Q}\sum_{r,i} i\, x_{jri} \quad (9)$$

$$\sum_{r,i} x_{jri} = 1, \qquad \forall j \in Q \quad (10)$$

$$\sum_{(j,0)\ni u} x_{jr0} + n_r(u) \leq \mathrm{cap}_r + z^A_{ru} \quad \forall r,\ u \in U_t \quad (11a)$$

$$\sum_{(j,i)\ni \sigma} x_{jri} + n_r(\sigma) \leq \mathrm{cap}_r + z^A_{r\sigma} \quad \forall r,\ \sigma \in [t+1, t+H] \quad (11b)$$

$$\sum_{(j,i)\ni\sigma,\ j\in Q^{\mathrm{def}}} x_{jri} + n_r(\sigma) \leq \mathrm{cap}_r - \rho_r + z^B_{r\sigma} \quad \forall r,\ \sigma \in [t+1, t+H] \quad (12)$$

변수는 $0 \leq x_{jri} \leq 1$, z^A, $z^B \geq 0$이다. $(j, i) \ni \sigma$는 작업 j가 리전 r에서 i 슬롯 뒤에 시작해 d_j 동안 도는 실행이 슬롯 σ와 겹친다는 뜻이고(σ는 슬롯 번호로, 작업의 시작 시각 τ_j와 다르다), Q^{def}는 대기 작업 가운데 미룰 수 있는 작업의 집합이다. $n_r(\sigma)$는 슬롯 σ를 조금이라도 덮는 확정 작업의 수, $n_r(u)$는 순간 u에 실행 중인 작업 수로 둘 다 장부에서 읽는다. $U_t = \{t\} \cup \{$이번 슬롯에 도착한 작업의 $s_j\}$는 지금 슬롯의 시작 순간 집합이고, $(j, 0) \ni u$는 지금 시작하는 작업 j의 실행 $[\max(t, s_j), \max(t, s_j)+d_j)$가 순간 u를 포함한다는 뜻이다. z^A의 첨자 κ는 식 (11a)의 $u \in U_t$와 식 (11b)의 σ를 함께 가리킨다. 식 (10)은 한 작업을 한 자리에만 둔다는 뜻이다. 식 (11a)(11b)는 식 (6)의 물리 상한을 지금 슬롯의 시작 순간과 미래 슬롯에 대해 적은 것이다. 식 (12)는 미룰 수 있는 작업이 미래 슬롯을 끝까지 채우지 못하게 ρ_r 자리를 비워 두는 예약으로, 모든 리전과 $\sigma \in [t+1, t+H]$에 대해 성립한다.

초과 변수의 벌점은 $M_A \gg M_B \gg 1$로 두어 벌점에 순서를 만든다. 물리 초과를 가장 무겁게, 그다음 예약 침범을, 그 안에서 탄소와 지연을, 마지막으로 ε항의 이른 시작을 본다. 상한과 점유가 정수라 정수해에서는 z도 정수로 떨어지므로, 이 순서가 엄밀히 지켜지려면 M_B가 대기 작업의 실행 시간 합 $\Sigma_{j\in Q} d_j$와 ε항의 최댓값을 더한 것보다 커야 하고, M_A도 한 작업을 옮길 때 함께 바뀌는 예약 침범의 크기만큼 M_B보다 커야 한다. 실제로는 M_B를 가장 긴 실행 시간의 2배(합성 워크로드 12, Azure 1,438.5), M_A를 그 100배로 두었으며 이는 보장이 아니라 실용적 선택이다. 합성 워크로드 주 실험의 최종해에서는 두 초과 변수가 1년 8,760 슬롯 어디에서도 양수가 되지 않았다. Azure에서는 예약 침범 z^B가 무릎점에서 슬롯의 52.6%(4,605/8,761), α=1에서 90.5%(7,949/8,780)에서 양수이지만, M_B를 0.1~10배로 바꿔도 절감은 46.14~46.22%로 같아(위반은 모두 0) 결과가 벌점 크기에 매이지 않는다. 4.7절의 예약 비용은 벌점이 아니라 예약 행 자체에서 온다. 초과 벌점은 행마다 매기며 초과가 이어진 시간으로 가중하지 않는다. ε은 10^{-4}로 두어 대개 비용이 같은 후보 사이에서 이른 시작을 고르게 하지만, 분 단위 작업에서는 탄소 차보다 클 수 있다.

용량 행은 두 가지 해상도로 센다. 미래 슬롯($\sigma \geq t+1$)은 슬롯을 조금이라도 덮는 작업을 한 자리로 보는 보수적 근사다. 매 슬롯 다시 풀리므로 확정 시점에 바로잡히지만, 그 전까지는 다른 작업의 선택에 영향을 준다. 채점 구간에서 실행 시간이 1시간 미만인 작업 111,142건 가운데 제출 슬롯보다 뒤 슬롯에 시작해 이런 자리에 올랐던 것은 13,034건(11.7%, 중간 계획에서 센 값)이다. 지금 슬롯만은 식 (11a)처럼 작업이 시작하는 순간마다 한 행을 두고, 그 순간에 돌고 있는 작업을 실제 시각 그대로 센다. 그래야 1초짜리 작업 수십 건이 한 시간 자리를 통째로 차지하는 가짜 포화가 생기지 않고, 4.1절의 위반 정의와 같은 기준으로 센다.

식 (9)~(12)는 먼저 변수를 0~1 연속으로 두고 선형 계획으로 푼다. 어느 작업도 슬롯 경계를 넘지 않고 지금 슬롯도 슬롯 단위로 센다면 식 (10)은 작업별 분할이고 (11b)(12)는 같은 (r, σ)에서 포개지는 집합이라 두 라미나 집합족의 합이 되어 제약 행렬이 완전 단모듈이고[21], 선형 계획의 해가 곧 정수해다[22, 23]. 실제로는 슬롯 경계를 넘는 작업과 지금 슬롯의 순간 행이 있어 이 조건이 거의 모든 슬롯에서 깨지지만, 분수해가 난 슬롯은 3.9%뿐이었고(4.9절) 그 슬롯만 정수 계획으로 다시 푼다. 풀고 나면 i=0인 작업만 확정해 리전과 시작 시각을 장부에 올린다. i≥1로 배정된 작업은 계획일 뿐이며 다음 슬롯에 다시 Q에 들어간다. 그래서 어떤 미래 슬롯도 미리 예약되지 않지만, 지금 시작하는 긴 작업이 미래 슬롯을 덮는 것은 식 (11)(12)가 $n_t(\sigma)$로 센다.

상한 cap_r은 본래 설비 규모에서 와야 하지만 클라우드 사업자는 리전별 설비 규모를 공개하지 않는다. 그래서 상한을 하나의 상수로 유도하지 않고 실험 변수로 둔다. 단위는 모든 작업을 출발 리전에서 즉시 실행했을 때의 리전별 최대 동시 실행 수, 곧 기준 피크이고, 본문은 그 1.25배를 대표 상한으로 쓰며 0.5배에서 3배까지 바꾼 결과를 4.7절에 둔다.

예약 ρ_r도 자료에서 정한다. 미룰 수 없는 작업은 도착한 슬롯에 바로 돌아야 하므로 막을 방법이 없고, 슬롯 σ가 시작될 때 이전 슬롯에 시작해 아직 돌고 있는 작업이 상한까지 차 있으면 σ에 도착하는 작업은 들어갈 자리가 없다. 식 (12)는 미래 슬롯으로 넘어가는 미룰 수 있는 작업을 $\mathrm{cap}_r - \rho_r$ 아래로 묶어 매 슬롯 시작에 ρ_r 자리를 남긴다. 지금 슬롯의 도착은 슬롯 시작 시점에 모두 알므로 $\sigma = t$에는 예약을 두지 않는다. ρ_r의 값은 평가 구간 앞의 준비 구간(1월 1~7일)에서 잰 미룰 수 없는 작업의 리전별 최대 동시 실행 수이며, 리전별 최댓값 가운데 가장 큰 값을 전 리전에 같게 쓴다. 합성 워크로드는 3, Azure 워크로드는 13이다. Azure의 13에는 이전 슬롯에서 넘어와 아직 도는 긴 VM이 들어가 식 (12)의 $n_t(\sigma)$와 두 번 세어지므로 과대값이며, 같은 슬롯에 새로 도착하는 몫만 재면 리전당 5~7이다(합성은 미룰 수 없는 작업이 짧아 이 몫이 거의 없다). 평가 구간의 자료로 정하지 않았으므로 결과를 보고 고른 값이 아니다. ρ_r을 0에서 6까지 바꾼 결과는 4.7절에 있다.

예약이 있어도 초과는 날 수 있다. 마감이 코앞인 작업이 상한보다 많이 몰리면 식 (11)의 z^A가 양수가 된다. 그래서 스케줄러가 하는 일은 용량을 강제하는 것이 아니라 초과에 가장 큰 벌점을 두어 최소화하는 것이며, 생긴 초과는 건수와 시간으로 모두 보고한다(4.4절). 스케줄러는 세 가지를 보장한다. 첫째, 후보 시작 시각이 애초에 $D_j - d_j$ 이하로만 잡히고 여유가 한 슬롯보다 적게 남으면 i=0만 허용되므로, 마감을 넘긴 배정은 탐색 공간에 없다. 둘째, 슬롯이 하나 지날 때마다 남은 여유는 정확히 1씩 줄고 시작 값이 유한하므로 유한한 슬롯 안에 i=0만 남는다. 어떤 작업도 무한히 연기되지 않는다. 셋째, 초과 변수가 있어 식 (9)~(12)는 언제나 해를 가진다.

3.4 로드밸런서와 스케줄러의 결합

식 (9)~(12)는 리전과 시작 슬롯을 한 문제에서 정한다. 둘을 나누어 정하는 설계와 비교하면 이렇게 한 까닭이 드러난다. 작업이 도착할 때 리전을 정하고 시작 시각은 그 리전 안에서 나중에 고르며 용량은 입장 때만 확인하는 2단계 설계에서는, 각 작업이 상한을 보지 않고

제 창 안에서 예측 탄소가 가장 낮은 쪽의 시각을 고른다. 그 시각이 지금인 작업만 빈자리만큼 들어가고 나머지는 다음 슬롯에 다시 고른다. 같은 리전의 작업들은 같은 예측을 보므로 같은 골을 고르고, 예측 곡선의 골이 뾰족할수록 줄이 길어진다. 줄에 선 채 여유가 다한 작업은 자리가 없어도 들어가므로 위반이 생긴다(4.4절). CAST에서는 용량이 입장 조건이 아니라 모든 대기 작업이 함께 보는 제약이어서, 한 골에 상한보다 많이 몰리는 배정은 식 (9)의 초과 항 때문에 애초에 비싸다.

남은 하나는 식 (7)의 가중치 α다. 지연은 예산 안에서도 낮을수록 좋다. 원격 리전을 점유하는 동안 자료가 오가고 장애에 노출되므로, 지연 비용은 그 점유 시간에 비례하게 둔다. 그래서 탄소와 지연은 서로 다른 두 목적이고, α가 둘의 맞바꿈을 정한다. α=1이면 지연을 보지 않고 탄소만 줄이며, α=0이면 출발 리전에서 즉시 실행하는 것과 같다. CAST는 가중치를 손으로 정하지 않는 기본값으로, 파레토 곡선이 가장 크게 꺾이는 점이라는 무릎점[24] 대신 이상점에 가장 가까운 후보, 곧 타협해[25]로 α를 슬롯마다 고르고 이를 편의상 무릎점이라 부른다. 0부터 1까지 0.1 간격의 후보 11개, 곧 $\mathcal{A}$ = {0, 0.1, …, 1}의 각 α에 대해 식 (9)~(12)를 풀어 그 배정의 실행 시간 가중 평균 지연 $\tilde{\lambda}_\alpha = \Sigma_j\, d_j\, \ell(o_j, r(j)) \,/\, \Sigma_j\, d_j$와 예측 배출량 $\tilde{e}_\alpha$를 얻고, 두 축을 후보들 사이에서 각각 0에서 1로 정규화한 뒤 원점에 가장 가까운 후보의 α와 배정을 택한다.

$$\alpha^* = \operatorname*{argmin}_{\alpha \in \mathcal{A}} \sqrt{\tilde{\lambda}_\alpha^2 + \tilde{e}_\alpha^2} \qquad (13)$$

여기서 $\tilde{\lambda}_\alpha$와 $\tilde{e}_\alpha$는 각각 0~1로 정규화한 평균 지연과 예측 배출이다. 이 규칙은 다중목적 최적화에서 이상점까지의 거리로 타협해를 고르는 방법이다. 물리 초과가 생기는 후보가 있으면 초과가 최소인 후보들 안에서만 고르고, 거리가 같으면 작은 α를 택한다. 홈 리전에 고정된 경우처럼 모든 후보의 지연이 같거나 배출이 같아 한 축이 한 점으로 줄면 α=1로 둔다. 배출 축만 한 점으로 줄 때도 α=1을 두므로, 그 슬롯에서는 지연이 가장 짧은 배정을 고르지 못할 수 있으며, 37일 893 슬롯을 세어 보니 그런 슬롯은 없었다(대표 상한 17). 두 축을 모두 실행 시간으로 가중하는 것은 목적함수 식 (1)(2)와 같은 자로 재기 위해서다. 건수로 평균하면 1초짜리 작업 한 건이 6시간짜리 작업 한 건과 같은 표를 갖는다.

리전과 시각을 한 문제로 둔 것이 배정을 어떻게 바꾸는지는 한 슬롯을 열어 보면 분명하다. 그림 4의 슬롯에서 한국을 출발한 작업 하나를 보면, α를 0에서 1까지 옮길 때 배정이 다섯 구간으로 갈린다(표 1). α가 0이면 한국에서 지금 시작하고(352 gCO₂/kWh),

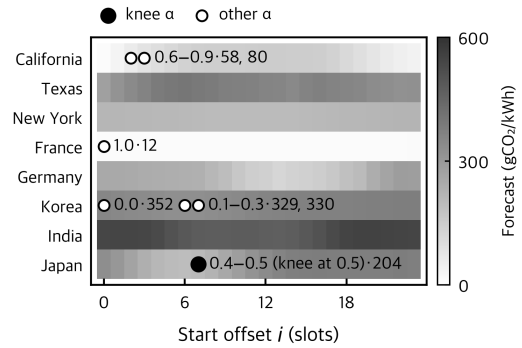


그림 4 한 슬롯에서 α에 따른 배정. 2025년 5월 26일 19시 48분(UTC)에 한국에서 제출된 k=1 작업 하나(실행 17분, 마감까지 14.5시간)이며, 칸은 리전 × 향후 24 슬롯이고 진할수록 예측 탄소집약도가 높다. ●는 무릎점(α=0.5)이 고른 칸, ○는 다른 α가 고른 칸이며 옆 숫자는 그 칸을 고른 α의 범위이고, · 뒤 숫자는 그 칸의 예측 탄소집약도(gCO₂/kWh)다.

Fig. 4 Assignment by α at one slot. One k=1 job submitted in Korea at 19:48 UTC on 26 May 2025 (17 min run, 14.5 h to deadline); cells are region × next 24 slots, darker for higher forecast carbon intensity. The filled dot is the cell chosen at the knee (α=0.5), hollow dots those chosen at other α, labelled with the range of α that chose them, followed after · by the cell's forecast intensity (gCO₂/kWh).

표 1 가중치 α에 따른 배정 변화. 그림 4와 같은 슬롯과 작업(출발 리전 한국)이며, 시작은 지금부터 몇 슬롯 뒤인지, 탄소는 예측 강도(gCO₂/kWh), 지연은 ms다.

Table 1 Assignment change by weight α. Same slot and job as Fig. 4 (origin: Korea); start is in slots from now, carbon is forecast intensity in gCO₂/kWh, latency in ms.

α	Region	Start	Carbon	Latency
0.0	Korea	0	352	0
0.1–0.3	Korea	6–7	329–330	0
0.4–0.5	Japan	7	204	30
0.6–0.9	California	2–3	58–80	134
1.0	France	0	12	238

0.1~0.3이면 한국에 남되 6~7시간 뒤로 미뤄 329~330으로 낮춘다. 0.4~0.5에서는 지연 30 ms를 들여 일본으로 옮기고 7시간 뒤에 시작해 204로, 0.6~0.9에서는 캘리포니아로 옮겨 2~3시간 뒤에 58~80으로, 1.0에서는 프랑스에서 즉시 시작해 12로 간다. 리전만 바뀌는 것이 아니라 리전마다 고르는 시작 시각도 다르며, α 하나가 로드밸런서와 스케줄러의 선택을 함께 움직인다. 무릎점은 이 슬롯에서 0.5를 골라 지연 30 ms로 예측 강도를 42% 낮춘다.

이 작업 하나만 보면 α가 커질수록 탄소가 단조롭게 줄지는 않는다. 같은 슬롯의 다른 작업 29건과 캘리포니아의 자리를 나눠 쓰기 때문이다. α=0.7에서는 캘리포니아의 t+2 자리를 여섯 건에 배정되어 이 작업이 t+3으로 밀렸다. 작업마다 따로 골랐다면 이 작업은 예측 강도가 더 낮은 t+2(58)를 골랐을 것이다(t+3은 80).

이 규칙은 가중치 값 대신 후보 격자, 정규화, 거리 규칙을 정해 두고 슬롯마다 α를 고른다. 그래서 무릎점은 곡선의 모양에 따라 움직인다. 예측된 탄소 곡선의 골이 뾰족할수록 탄소 축의 정규화가 달라져, 같은 슬롯에서도 지연 쪽 α를 고른다(4.5절). 슬롯들이 실제로 고른 α의 분포는 4.5절에 있다.

무릎점은 배출이 가장 낮은 지점이 아니라, 지연을 함께 줄이는 쪽의 한 점이다. 목적식과 같은 자(실행 시간 가중 지연)로 짝지은 같은 지연의 고정 α보다 합성 워크로드의 아홉 조건 가운데 여덟에서 더 배출하고 미룸은 짧으므로(4.5절), 설계의 기여가 아니라 운용 규칙으로 쓴다. 알고리즘 1은 세 모듈이 한 슬롯에서 하는 일을 차례로 적은 것이다.

비교를 위해 여덟 가지를 함께 잰다. 첫째는 작업이 도착하는 슬롯에서 로드밸런서가 리전을 정해 고정하고, 그 뒤로는 스케줄러가 시작 슬롯만 같은 문제로 고르게 한 것이다. 비용, 상한, 예약, 예측, 가중치 규칙은 모두 같고 리전을 정하는 시점만 다르며, 4장에서는 분리형이라 부른다. 둘째는 같은 배정에서 식 (11)(12)를 지운 것으로, 저탄소 시각에 몰려도 아무것도 막지 않는다. 용량을 지운 기준점이며 용량 무시라 부른다. 셋째는 같은 비용, 상한, 무릎점 규칙을 쓰되 선형 계획 대신 남은 여유가 적은 작업부터(LLF) 각자 가장 싼 (리전, 시작 슬롯)을 차례로 잡게 한 것으로, 탐욕(greedy)이라 부른다. 넷째는 위에서 말한 2단계 설계로, 예측, 물리 상한, 1시간 선견, 채점은 CAST와 같다. 다른 점은 리전을 작업이 도착할 때 슬롯 단위 정수 계획(가중치는 슬롯마다 무릎점)으로 정해 고정하되, 이 리전 배정은 별도로 구현한 2단계 로드밸런서(리전당 상한은 기준 피크의 1.2배에 내부 여유 0.8을 곱한 12)가 LSTM 예측으로 한 번 계산한 것을 모든 예측기와 내부 상한에 똑같이 쓰고(그래서 평균 지연이 모두 같다), 시작 시각은 그 리전 안에서 작업마다 후보 창의 예측 탄소를 최솟값–최댓값으로 0~1로 바꾼 값과 창 길이 대비 미룸의 비율을 $(6-k)/5$ 대 $1-(6-k)/5$로 더한 점수가 가장 낮은 시각을 따로 고르며(첫 시각은 예측 대신 그 시각의 실측을 쓴다), 그 시각이 지금인 작업만 남은 여유가 적은 순으로 내부 상한까지 들여보내고, 여유가 다한 작업과 미룰 수 없는 작업은 자리가 없어도 들여보낸다는 것이다. 다시 풀지 않고 예약도 두지 않으며, PuLP와 CBC로 푼다. 확인 시점 말고도 다른 점이 많아, 확인 시점만 다른 대조군은 아래 넷으로 따로 만든다. 다섯째는 Zanotto 등[9]처럼 작업이 도착한 슬롯에서 이미 정한 배정을 모두 보고 리전과 시작 슬롯을 한 번 정한 뒤 다시 풀지 않는 것으로, 도착 고정형(Arrival-fixed)이라 부른다. 원 설계보다 불리하지 않도록 한 슬롯에 도착한 작업을 묶어 CAST와 같은 무릎점 선형 계획으로 정한다. 여섯째는 도착 고정형과 같되 미래 슬롯의 용량을 이미 시작한 작업만 보고 확인하는 것으로, 시작 확인형이라 부른다. 확정했지만 아직 시작하지 않은 다른 작업의 계획을 보지 않는다는 한 줄만 도착 고정형과 다르다. 일곱째는 시작 확인형에 시작 때의 확인을 더한 것으로, 정한 시각이 오면 그 순간의 장부로 다시 확인해 자리가 있으면 시작하고 없으면 한 슬롯 미루며, 여유가 다하면 자리가 없어도 들인다. 입장 통제형이라 부른다. 입장 통제형과 같되 여유가 다해 들여야 할 때 정한 리전이 꽉 찼으면 그 순간 자리가 있는 리전 가운데 예측 탄소가 가장 낮은 곳으로 보내는 변형은 대피형이라 부른다. 여덟째는 CAST에서 미래 슬롯 행(식 (11b)(12))만 지운 것으로, 순간 확인 롤링이라 부른다.

알고리즘 1 결합 롤링 배정

Algorithm 1 Joint rolling assignment

Input: cap_r, ρ_r, H, forecasts $\hat{C}_r^{(t)}(\cdot)$, job stream
Output: $(r(j), \tau_j)$ for all j

$Q \leftarrow \emptyset$
for $t \leftarrow 0, 1, 2, \ldots$ do
  for each j with $\lfloor s_j \rfloor = t$ do $Q \leftarrow Q \cup \{j\}$
  for each $j \in Q$ do
    $h_j \leftarrow \lfloor \min(D_j - d_j,\ t + H) - t \rfloor$
    if j is non-deferrable or $(D_j - d_j) - t < 1$ then $h_j \leftarrow 0$
  for each $\alpha \in \{0, 0.1, \ldots, 1\}$ do
    $x^{\alpha} \leftarrow$ LP solution of (9)–(12)
    $(\bar{\lambda}_{\alpha}, \bar{e}_{\alpha}) \leftarrow$ duration-weighted mean latency and forecast emissions of x^{α}
  $\alpha^{*} \leftarrow$ knee point by (13) with the tie rules of Sec. 3.4; $x \leftarrow x^{\alpha^{*}}$
  if x is fractional then $x \leftarrow$ MILP solution of (9)–(12) at α^{*}
  for each $j \in Q$ with $x_{jr0} = 1$ do
    $r(j) \leftarrow r$; $\tau_j \leftarrow \max(t, s_j)$; record $[\tau_j,\ \tau_j + d_j)$ in ledger; $Q \leftarrow Q \setminus \{j\}$

3.5 탄소 회계

작업 j의 배출량은 실행 리전의 실측 탄소집약도를 실행 구간에 대해 적분한 값에 소비 전력을 곱한 값이며, 그 합이 식 (1)이다. 실측 계열이 1시간 해상도이므로 적분은 슬롯 단위로 근사한다. 실행이 걸치는 슬롯들의 평균 강도에 실행 시간을 곱한다.

$$E_j \approx P d_j \frac{1}{|T_j|} \sum_{\sigma \in T_j} C_{r(j)}(\sigma) \tag{14}$$

여기서 T_j는 시작 슬롯 $\lfloor \tau_j \rfloor$부터 $\lceil d_j \rceil$개 슬롯의 집합이다. 겹침 길이로 가중해 정확히 적분해도 절감은 무릎점 71.93%에서 71.80%로, α=1은 95.65%에서 95.64%로 움직일 뿐이다. 소비 전력은 작업당 1 kW 상수로 두고 유휴·냉방·내재 배출은 넣지 않으므로, 절감률의 분모는 작업이 실행 중에 쓰는 전력의 배출이다. 상대 비교를 위한 단순화이며, 그 덕에 배출량 차이는 전적으로 실행 리전과 시각에서 온다. 4장의 모든 정책과 대조군은 이 회계 하나로 채점한다.

4. 실험과 분석

4.1 실험 설정

평가는 8개 리전의 2025년 탄소집약도 1년치 위에서 수행한다. 이 값은 Electricity Maps가 발전원 구성과 배출계수로 계산해 사후에 확정한 CO_2 환산량(gCO_2eq/kWh)이며, 이하 예측과 구별해 실측이라 부르고 단위는 gCO_2/kWh로 적는다. 탄소집약도는 Electricity Maps API를 통해 1시간 해상도로 수집하였다[26]. Electricity Maps의 값은 수입과 수출을 따라간 소비 기준이며[27], 받은 자료에는 직접 배출과 전과정 가운데 어느 기준인지 표시가 없으나, 프랑스의 최솟값이 2 gCO_2/kWh여서 직접 배출 기준으로 보인다. 무탄소 전원에 전과정 배출(원자력 12, 재생 30 gCO_2/kWh)을 더한 근사(화석 상류 배출은 넣지 않은 한쪽 근사)로 배정은 그대로 두고 다시 채점하면 절감이 1.7~5.4%포인트 줄지만(CAST 무릎점 67.07%, α=1 90.28%, Azure 무릎점 43.17%) 설계의 순서와 용량 비용의 부호는 같다. 평균 계수의 결과는 프랑스가 받은 부하가 깨끗하게 공급된다는 가정에 기댄다. 프랑스로 옮긴 작업만 같은 시각 독일의 평균 강도로 세는 프랑스 한계 시나리오에서는 CAST 무릎점이 38.99%, α=1이 18.11%(시간만 18.78%), Azure 무릎점이 19.59%로 줄고, 같은 지연의 재풀이 이점도 사라진다(37.53% 대 37.55%). 캘리포니아로 옮긴 몫(무릎점 절감의 약 25%)과 홈 안의 시간 이동 몫은 평균 계수 그대로이므로, 이들까지 한계 배출로 세면 더 줄어든다. 이 시나리오는 프랑스로 간 몫만 바꾼 한쪽 대리다. 출발지, 목적지, 기준을 모두 화석 몫의 강도로 세는 대칭 대리에서는 절감이 36.15%(무릎점)로 줄지만 같은 지연의 비교 순서는 평균 계수와 같고(CAST 고정 α 34.84% 대 도착 고정형 33.78%, 대피형 33.30%), 시간 이동만은 0.42%로 거의 사라진다. 위반과 필요 용량은 회계와 무관해 바뀌지 않는다. 리전은 한국, 캘리포니아, 텍사스, 뉴욕, 프랑스, 독일, 인도, 일본이며, 각 리전에는 Electricity Maps의 다음 구역 자료를 썼다. 캘리포니아 US-CAL-CISO, 텍사스 US-TEX-ERCO, 뉴욕 US-NY-NYIS, 프랑스 FR, 독일 DE, 한국 KR, 일본 JP, 인도 IN이며, 지연 행렬의 행은 같은 순서로 Azure 리전 West US, South Central US, East US, France Central, Germany West Central, Korea Central, Japan East, Central India다. 미국 동부 리전은 탄소 자료로 뉴욕 전력망(NYISO) 구역을, 지연 자료로 Azure East US 행을 쓰며, 이후 뉴욕이라 부른다. 뉴욕에는 Azure 리전이 없어 버지니아의 East US를 썼으므로, 이 리전은 지연과 탄소가 서로 다른 곳(PJM과 NYISO)에서 온 혼합이다. LSTM의 학습(2021~2023년), 검증(2024년), 시험(2025년) 자료는 같은 수집 파일 묶음이며, 스케줄러가 쓰는 1월 8일부터의 2025년 실측은 시험 파일과 리전별 평균 절대차 0.27 gCO_2/kWh 안에서 같다. LSTM이 168시간을 입력받아 2025년 예측이 1월 8일 0시부터 나오므로, 1월 1~7일은 1월 8일의 하루 모양을 되풀이해 채운 준비 구간이고 그 기간에는 예측과 실측이 같다. 2024년 말 자료로 입력을 채우면 학습과 검증의 분할이 섞이므로 그렇게 하지 않았다. 시뮬레이션은 1월 1일부터 돌리되 모든 지표는 1월 8일 0시부터 연말까지 8,592시간에 대해 세고, 그 전에 제출된 작업은 채점에서 뺀다.

네트워크 지연은 8×8 리전 간 왕복 지연 실측 행렬(최댓값 244 ms, 비대각 최솟값 12 ms)을 Azure 통계[28]에서 가져왔다. 전 구간에서 고정이며, 부하에 따라 지연이 바뀌는 효과는 모델에 없다.

워크로드는 둘이다. 합성 워크로드는 총 146,000건이며 리전당 하루 50건이 제출되고, 채점 구간에 들어오는 것은 143,200건이다. 등급별 비율(k=1부터 3:1:1:2:3)은 공개 트레이스에 지연 등급 표시가 거의 없어(Azure 트레이스는 0.5%) 정한 합성 파라미터다. 지연 예산은 등급 안에서 로그균등으로, 실행 시간은 로그정규 분포에 상·하한을 두어 뽑는다(표 2). 도착에는 하루 주기가 있다. k=3~5는 현지 13시, k=1~2는 현지 2시를 봉우리로 하는 표준편차 3시간의 정규분포에서 70%가 오고 나머지 30%는 봉우리 반대쪽 12시간에 고르게 온다(봉우리를 한낮으로 옮긴 결과는 4.7절). 현지 시각은 일광절약시간 없는 고정 시차로 둔다. 미룰 수 없는 작업이 건수로 59.1%지만 작업시간으로는 1.3%라, 시간 이동의 여지는 작업시간 기준으로 거의 전부에 있다. 공간 이동은 식 (5)가 걸리지 않아 모든 작업에 열려 있다.

표 2 지연 등급별 예산과 실행 시간. 예산은 등급 안에서 로그균등, 실행 시간은 로그정규에 상·하한을 두어 뽑는다. 중앙값과 상·하한은 초 단위, s.d. (log)는 로그정규의 로그 표준편차이며 중앙값과 함께 생성 모수이고 건수는 1년 전체다.

Table 2 Latency budget and job duration by tier. Budgets are log-uniform within each tier; durations are log-normal and clamped (seconds); the median and s.d. (log), the standard deviation of the log, are generation parameters. Counts cover the full year.

k	Budget	Median d	s.d. (log)	Clamp	Jobs
1	12–24 h	5,400	0.9	300–21,600	43,830
2	2–8 h	1,800	0.9	60–14,400	14,458
3	2–10 min	180	0.8	5–1,800	14,716
4	15–60 s	30	0.7	1–300	29,244
5	1–5 s	5	0.6	1–60	43,752

두 번째 워크로드는 Azure의 2019년 VM 트레이스[29, 30]에서 만들었다. 29일씩 13장을 이어 붙여 1년을 만들되 장마다 다른 VM 11,600개를 뽑았고(추출률 0.49%), 마지막 장은 연말에서 잘려 146,016건이다. 실행 시간은 VM이 생성되어 삭제될 때까지의 간격 그대로이고(대부분 300초 단위, 5.9%는 150초), VM 크기는 쓰지 않고 모든 VM을 같은 전력과 같은 용량 한 칸으로 셌으므로 Azure의 숫자는 VM 시간으로 가중한 값이다. 도착 시각은 트레이스의 하루 모양을 그대로 따르되, 그 VM을 배정한 출발 리전의 현지 시각이라고 가정해 일광절약시간 없는 고정 시차로 UTC로 옮겼다. 기준 배출은 213,513.4 kg으로 합성의 7.5배이고, 이 워크로드의 %포인트는 이 값에 대한 차다. 트레이스에 없는 두 값은 모형으로 채웠다. 출발 리전은 여덟 리전에 고르게 무작위로 주고, 지연 등급은 트레이스가 지연 민감 여부를 표시한 0.5%만 그 표시를 따르며 나머지는 합성 워크로드와 같은 비율로 준다. 도착과 길이는 실측이고 지연 예산은 모형이다. 시각 값은 그 한 시간의 시작을 뜻하고(UTC, 2025년 1월 1일 0시가 $t=0$), 연말을 넘는 작업과 지평에는 마지막 시각의 값을 이어 쓴다.

두 워크로드는 실행 시간에서 서로 다른 끝을 덮는다. 실행 시간의 중앙값은 합성 81초, Azure 1,500초이고, 95 백분위수는 1년 전체 작업 기준으로 합성 13,074초, Azure 74,400초(20.7시간)다. 합성은 VM 트레이스가 300초 해상도라 보지 못하는 짧은 작업을, Azure는 하루를 넘는 긴 작업을 담는다. 총 작업시간은 Azure가 합성의 7.8배이고, 평균 동시 실행은 여덟 리전 합으로 합성 11.5건(총 상한의 8.4%), Azure 89.9건(26.1%, 1년 전체 기준)이며, 미룰 수 없는 작업이 차지하는 작업시간 비중은 합성 1.3%, Azure 51%로 정반대다.

등급 비율 3:1:1:2:3은 두 원칙을 따랐다. 첫째, 시간 이동이 가장 잘 듣는 등급 1과 지연에 가장 민감한 등급 5를 같은 30%로 두었다. 다만 지연 예산이 모두 왕복 지연보다 길어 공간 이동은 모든 작업에 열려 있으므로, 이 워크로드는 공간 축에 유리하다(4.3절의 공간만 61.57% 대 4.4절의 시간만 18.78%). 네트워크 지연에 상한을 두어 공간 이동을 막았을 때의 결과는 4.7절에 있다. 둘째, 1초에서 하루까지 다섯 자릿수에 걸친 지연 예산의 어느 구간도 비지 않도록 가운데 세 등급을 1:1:2로 얇게 두었다. 그 결과 합성 워크로드는 건수로는 짧은 작업이, 작업시간으로는 긴 작업이 대부분인 꼴을 갖는다. 15초 미만 작업은 건수의 32.2%지만 작업시간의 0.1%이고, 1시간 이상 작업은 건수의 22.4%로 작업시간의 85.2%를 차지한다. Google 트레이스에서도 대다수 작업이 몇 분 안에 끝나지만 전체 기간 내내 도는 작업이 2,000개를 넘고 전체의 7%인 생산 우선순위 작업이 나머지 전부보다 많은 자원을 쓴다[31]. 미룰 수 있는 작업의 비율을 20~60%로 바꾼 결과는 4.7절에 있다.

상한은 3.3절대로 실험 변수다. 기준 피크는 채점 구간 작업을 홈 리전에서 즉시 돌렸을 때의 리전별 최대 동시 실행으로, 합성 워크로드에서 캘리포니아 11, 텍사스 10, 뉴욕 10, 프랑스 10, 독일 11, 한국 12, 인도 13, 일본 12이고, Azure 워크로드에서는 가장 큰 것이 34다. 상한 자체는 이 값의 0.5~3배로 바꿔 본다(4.7절). 대표 상한은 가장 큰 기준 피크의 1.25배를 올림한 값으로 합성 17, Azure 43이며, 모든 리전에 같게 주므로 리전마다 자기 피크의 1.3~1.7배가 된다. 어떤 정책이든 채점은 이 물리 상한으로 한다. CAST가 안에서 쓰는 예약 ρ는 채점 기준이 아니다.

지표는 다음과 같다. 총 배출량은 식 (1)을 3.5절의 회계로 잰 값이고, 절감률은 기준 대비 백분율이다. 용량 위반은 상한을 넘긴 배정 건수다. 작업 j가 시작하는 순간 같은 리전에서 이미 돌고 있는 작업이 상한 이상이면 j를 위반으로 센다. 같은 시각에 시작한 작업은 서로를 보고, 같은 시각에 끝나는 작업은 먼저 뺀다.

강제 시작은 여유가 한 슬롯 안으로 줄어 지금 시작해야 하는 작업 수다. 초과 시간은 리전별로 동시 실행이 상한을 넘은 시간의 합이다. 필요 용량은 그 정책의 배정이 만든 리전별 최대 동시 실행 수의 최댓값으로, 그 정책이 위반 없이 돌려면 필요한 상한이다. 채점 상한과 무관한 값이라 선행 정책 비교의 자로 쓰며, 1초짜리 순간 피크의 영향을 가리기 위해 시간 가중 99.9 백분위수를 함께 적는다. 이 밖에 마감 위반 건수, 공간 이동이 더한 왕복 지연의 평균과 95 백분위수, 제출에서 시작까지 미룬 시간을 잰다.

예측 곡선의 골의 깊이는 예측의 하루 최대−최소를 실측의 하루 최대−최소로 나눈 값(납작함)으로 잰다. 몰림, 곧 여러 작업이 같은 깨끗한 시각을 함께 노리는 정도는 세 값으로 설명한다. 리전-일마다 실측 탄소가 가장 낮은 1시간의 최대 동시 실행을 평균한 값, 그 시각에 동시 실행이 상한에 닿은 리전-일의 비율, 미룬 작업 가운데 그날 탄소가 가장 낮은 세 시각에 시작한 작업의 비율이다. 이 셋은 몰림의 모양을 보일 뿐이며, 몰림이 해로운지는 절감과 위반으로 가린다.

무릎점과 고정 α의 비교에는 같은 지연 짝을 쓴다. 무릎점의 평균 지연과 약 0.5 ms 안에서 같은 지연을 내는 고정 α(Azure 짝은 0.58~0.70 ms로 이를 조금 넘는다)를 이분법으로 찾아 1년 전체를 돌린 실행이다. 두 실행의 차는 무릎점이 더 배출한 양으로 적고, 7일 묶음을 되뽑아 2,000번 다시 계산한 블록 부트스트랩의 95% 신뢰구간이 0보다 위에 있으면 무릎점이 뒤진다고, 0을 포함하면 차이가 없다고, 0보다 아래에 있으면 이긴다고 판정한다. 이 신뢰구간은 한 실행 안의 주 단위 변동을 잰 것이며, 워크로드를 다시 뽑는 변동은 시드를 바꾼 다섯 벌의 실행으로 따로 잰다(4.7절). %포인트 차는 반올림 전 kg 차를 기준 배출로 나눈 값이라, 반올림한 절감률끼리 뺀 값과 0.01 다를 수 있다.

실험은 Python 3.14.7과 SciPy 1.18.0의 HiGHS 풀이기로 Apple M1 Max(성능 코어 8, 효율 코어 2, 메모리 32 GB, macOS 15.7.7)에서 돌렸다. 4.4절의 2단계 설계는 PuLP 3.3.2와 CBC 2.10.3으로 같은 기계에서 돌렸다. 모든 표의 숫자는 작업별 기록에서 별도 스크립트로 다시 계산해 대조했다.

본 연구의 코드, 합성 워크로드 생성기, Azure 트레이스 변환 스크립트, 실행 설정은 심사 뒤 공개한다. 탄소집약도 자료는 Electricity Maps 이용 약관에 따라 재배포하지 않고 전처리·학습 코드만 공개하며, 수집 스크립트는 남아 있지 않다.

4.2 예측기 평가

예측기는 값을 얼마나 맞히는가와 그 예측이 배정을 얼마나 잘 이끄는가로 나누어 본다. 2025년 시험 연도 전 지평의 평균 절대 오차는 여덟 리전 평균 26.10 gCO_2/kWh로, 직전값을 그대로 쓰는 예측(48.41)과 168시간 전 값을 쓰는 예측(35.46)보다 낮고 24시간 전 값을 쓰는 예측(SN-24h, 25.27)과는 비슷하다. 이 차는 인도 하나에서 온다. 인도의 하루 변동(최댓값−최솟값) 월평균이 2024년 11월부터 거의 두 배가 되어 학습 연도와 하루 모양이 달라졌고(인도만 LSTM 30.54 대 11.48), 인도를 빼면 LSTM이 25.47로 24시간 전 값(27.24)보다 낮다. 이 변화는 망의 변화라기보다 자료 산정 방식의 변화로 보인다(추정, 일본도 2024년 1월에 비슷한 변화가 보인다). 연평균이 학습 연도에서 시험 연도로 갈수록 크게 내려간 리전이 많아(프랑스 41.3, 14.8, 13.6 gCO_2/kWh) 학습과 시험의 수준 차이가 크며, 이것이 아래에서 보는 LSTM의 얕은 골의 한 까닭일 수 있다. 미래 기상 실측을 입력으로 쓰는 세 리전(캘리포니아, 텍사스, 독일)의 예측을 기상 입력이 없는 24시간 전 값으로 바꾸어도 CAST의 절감은 무릎점에서 0.37%포인트, α=1에서 0.01%포인트 움직이고 위반은 0이다(캘리포니아만 바꾸면 무릎점이 지연 쪽으로 조금 더 물러서 71.08%).

배정에 미치는 효과는 다르다. 예측기를 24시간 전 값, LSTM, 완전예지로 바꾸어 CAST를 돌린 결과가 표 3에 있다. α를 0.5로 고정하면 절감이 61.82%, 62.99%, 63.46%로 예측이 좋을수록 높다. 평균 절대 오차가 LSTM보다 조금 낮은 24시간 전 값이 절감은 1.17%포인트 낮으므로, 배정 결과를 정하는 것은 값의 오차만이 아니다. 곡선의 모양을 재면 24시간 창 안의 순위상관은 LSTM 0.629, 24시간 전 값 0.677이고, 예측 최저 시각이 실측 최저 시각과 맞는 비율은 21.5%와 30.3%로 오히려 24시간 전 값이 높다. 둘이 크게 갈리는 것은 골의 깊이다. 예측한 하루 최대−최소를 실측의 것으로 나눈 비가 24시간 전 값은 1.001, LSTM은 0.647로, LSTM은 골을 실측의 65% 깊이로 납작하게 그린다. 탄소만 보는 α=1에서는 95.59%, 95.65%, 95.87%로 차이가 0.28%포인트 안이다. 무릎점에서는 64.44%, 71.93%, 67.20%(평균 지연 39.2, 47.8, 40.7 ms)로 순서가 뒤집히는데, 이는 예측의 질이 아니라 무릎점 규칙이 곡선의 모양에 따라 다른 α를 고르기 때문이며 4.5절에서 가른다. 어느 예측기에서든 CAST의 위반은 0이다. 지금 슬롯의 첫 한 시간에 사후 확정값을 쓰는 탄소 쪽 선견을 없애고 한 시간 앞서 발행된 예측만 쓰면 위반은 그대로 0이고 α=1은 0.17%포인트, Azure 무릎점은 0.52%포인트 낮아진다. LSTM 무릎점은 2.42%포인트 낮은 69.50%가 되는데(24시간 전 값은 오히려 1.01%포인트 높은 65.45%), 무릎점이 고르는 α가 달라진 몫이 크다(지연 47.85 ms에서 45.89 ms).

표 3 예측기와 설계에 따른 절감률(%)과 평균 지연(ms). 괄호는 상한 17을 넘긴 배정 건수이고 —는 돌리지 않은 칸이다. 2단계의 η는 내부 상한을 물리 상한의 몇 배로 두었는지이며, η=1.0이면 내부 상한이 물리 상한과 같고, η=0.83은 내부 상한 14로 CAST의 미래 슬롯 상한 17−ρ와 같다. Arrival-fixed는 도착 고정형(3.4절)이다. 탄소 쪽 1시간 선견을 없애면 LSTM의 CAST 무릎점은 69.50%다(4.2절). 표의 영문 이름은 Oracle이 완전예지, Two-stage가 2단계, Decoupled가 분리형, Capacity ignored가 용량 무시다.

Table 3 Savings (%) and mean latency (ms) by forecaster and design. Parentheses give assignments past the ceiling of 17; a dash marks a run not performed. For the two-stage design, η is the internal ceiling as a fraction of the physical one; at η=1.0 the two coincide, and η=0.83 gives an internal ceiling of 14, equal to CAST's future-slot ceiling 17−ρ. Arrival-fixed is the arrival-fixed control of Section 3.4: each job's region and start are fixed once in its arrival slot against all existing plans and never re-solved. Without the one-hour carbon lookahead, CAST at the knee with LSTM saves 69.50% (Section 4.2).

Design	SN-24h	LSTM	Oracle
Two-stage, η=0.8	63.04 (0) / 36.2	63.78 (6) / 36.2	64.38 (19) / 36.2
Two-stage, η=0.83	—	64.31 (6) / 36.2	64.92 (31) / 36.2
Two-stage, η=1.0	64.54 (125) / 36.2	65.26 (18) / 36.2	66.02 (152) / 36.2
Arrival-fixed, knee	—	64.28 (0) / 41.2	64.23 (0) / 40.1
Arrival-fixed, α=1	—	93.03 (0) / 120.3	—
Decoupled, knee	43.77 (0) / 19.5	48.88 (0) / 23.4	46.62 (0) / 19.9
Decoupled, α=1	95.57 (0) / 119.9	95.34 (0) / 120.0	95.85 (0) / 119.9
CAST, fixed α=0.5	61.82 (0) / 32.7	62.99 (0) / 33.9	63.46 (0) / 32.6
CAST, knee	64.44 (0) / 39.2	71.93 (0) / 47.8	67.20 (0) / 40.7
CAST, α=1	95.59 (0) / 120.0	95.65 (0) / 120.0	95.87 (0) / 119.9
Capacity ignored, knee	—	73.39 (24,993) / 47.8	69.25 (33,964) / 41.5
Capacity ignored, α=1	—	95.97 (46,354) / 120.4	—

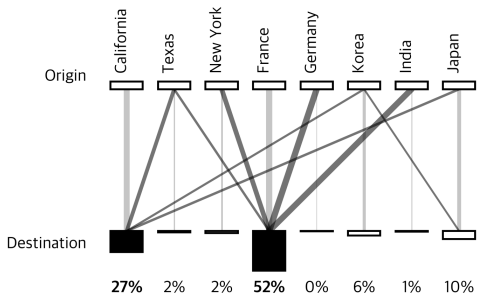


그림 5 로드밸런서가 만든 리전 간 흐름. CAST(무릎점)의 채점 구간 143,200건에서 옮긴 물량의 93.9%를 차지하는 흐름 8개이며, 회색 세로선은 홈에 남은 몫, 선 굵기는 옮긴 건수에 따라 굵어지고, 아래 상자 높이는 리전별로 실행된 몫에 정비례하며(몫이 2% 남짓 이하인 리전은 납작한 선으로 보인다) 검은 칠은 그 몫이 20%를 넘는 리전이다.

Fig. 5 Inter-region flow created by the load balancer. The 8 flows carrying 93.9% of moved jobs among the 143,200 scored jobs under CAST (knee α). Gray verticals are the share kept at the origin, line width grows with the number of jobs moved, lower box heights are strictly proportional to each region's share of executions (regions at about 2% or less appear as flat lines), and black fill marks regions above 20%.

4.3 로드밸런서 평가

본 연구의 일곱 방식을 같은 회계와 같은 상한 17로 비교하였다. 기준은 홈 리전 즉시 실행, 시간만은 홈을 고정하고 스케줄러만 쓴 것, 공간만은 시각을 고정하고 로드밸런서만 쓴 것이며, 분리형, CAST, 용량 무시, 탐욕은 3.4절에서 정의했다. 이 일곱 이름과 3.4절의 2단계, Arrival-fixed를 이후 모든 표와 그림에서 그대로 쓴다. 각 방식은 무릎점과 α=1 두 가중치로 돌렸고 결과는 표 4에 있다.

로드밸런서만으로도 절감의 대부분이 나온다. 공간만은 무릎점에서 61.57%(40.4 ms), α=1에서 93.96%(120.1 ms)를 줄이며, 두 경우 모두 최대 동시 실행이 상한 17을 넘지 않는다. 리전 상한을 보지 않는(자체 서버 용량은 사실상 묶이지 않는) CASPER는 지연이 120.4 ms로 같은 α=1 쪽에서 95.30%로 1.34%포인트 더 줄이지만, 그 배치는 리전 하나가 동시에 29건을 돌려야 성립한다(4.8절). 다만 그 1.34%포인트에는 용량을 지킨 대가와 함께 목적함수와 구현의 차이도 섞여 있다. 제약 두 줄만 다른 짝은 아래의 용량 무시다. 공간 이동이 실제로 만든 흐름은 그림 5에 있다. CAST는 채점 구간 작업의 58.4%인 83,665건을 출발 리전 밖에서 돌렸고, 리전별로 실행된 몫은 프랑스 51.6%, 캘리포니아 27.3%로 두 리전이 79.0%를 차지한다. 가장 큰 흐름은 독일에서 프랑스(옮긴 물량의 21.3%)와

표 4 일곱 방식의 절감률, 평균 지연, 최대 동시 실행. 각 칸은 무릎점 / α=1이며, 최대 동시 실행은 리전별 최댓값 가운데 가장 큰 값으로 4.8절의 필요 용량과 같은 자다. 상한은 17이며, CAST·분리형·용량 무시 행은 표 3의 LSTM 열과 같은 실행이고, 시간만은 지연 축이 없고 탐욕은 무릎점에서만 돌려 한 값만 적었다. 무릎점끼리의 차에는 설계마다 다른 α가 섞여 있으며, 같은 α에서의 설계 차는 표 5에 있다.

Table 4 Savings, mean latency and peak concurrency of the seven methods. Each cell is knee / α=1; Req. cap. is the largest per-region maximum concurrency, the same measure as the required capacity in Section 4.8. The ceiling is 17; the CAST, decoupled and capacity-ignored rows are the same runs as the LSTM column of Table 3; Temporal-only has no latency axis and Greedy was run at the knee only, so each has a single value. Differences between knees mix in each design's different α; same-α design differences are in Table 5.

Method	Savings (%)	Lat. (ms)	Req. cap.
Baseline	0 / 0	0 / 0	13 / 13
Temporal-only	18.78 / —	0 / —	17 / —
Spatial-only	61.57 / 93.96	40.4 / 120.1	17 / 17
Decoupled	48.88 / 95.34	23.4 / 120.0	17 / 17
CAST	71.93 / 95.65	47.8 / 120.0	17 / 17
Capacity ignored	73.39 / 95.97	47.8 / 120.4	63 / 117
Greedy	71.08 / —	47.6 / —	17 / —

인도에서 프랑스(20.2%)이며, 텍사스와 일본, 한국의 작업은 주로 캘리포니아로 간다.

이용률이 8.4%인 이 워크로드에서 CAST가 용량을 지키는 비용은 작다. 용량 무시는 무릎점에서 73.39%, α=1에서 95.97%로 CAST보다 1.47%포인트와 0.31%포인트(예약 3일 때) 더 줄이지만, 최대 동시 실행이 63건과 117건으로 상한의 3.7배와 6.9배이고 상한을 넘긴 배정이 24,993건과 46,354건이다. 넘친 자리는 거의 모두 로드밸런서가 작업을 모으는 리전에서 난다(4.6절). α=1의 CAST는 작업시간의 99.9%를 프랑스에서 돌려(무릎점은 프랑스 49.2%, 캘리포니아 31.2%, 일본 10.3%) 서론의 Sukprasert 정책과 같은 모습이며, 다른 점은 상한 안에서 그렇게 한다는 것이다(필요 용량 17 대 129). 그래서 α=1에서의 1%포인트 미만 차(이 0.31%포인트, 표 5의 α=1 칸, CASPER와의 차)는 프랑스 평균 배출계수로 약 1 gCO_2/kWh 수준의 차로 읽어야 한다. 작업당 1 kW 가정에서 CAST는 1년에 무릎점에서 약 20.6 t, α=1에서 약 27.4 t을 덜 내보낸다. 이 회계는 리전 사이로 옮긴 작업의 전송 에너지를 넣지 않았다. 옮긴 작업마다 데이터 1 GB를 보내는 데 드는 0.06 kWh[32]를 목적지 리전의 탄소집약도로 더해도, CAST의 절감은 무릎점에서 71.93%가 71.16%로, α=1에서 95.65%가 95.20%로 1%포인트 안에서 줄고, 비교한 아홉 실행(표 4에서 기준과 탐욕을 뺀 실행)의 순서는 바뀌지 않는다. 전송 에너지를 출발지의 실행 구간 평균 탄소집약도로 세면 무릎점은 66.01%(5.92%포인트 감소), α=1은 87.65%(8.00%포인트 감소)로 더 줄지만 순서는 여전히 같고, 작업이 긴 Azure에서는 무릎점이 46.14%에서 45.40%로 줄 뿐이다. 출발지로 셀 때 작업당 약 9 GB면 CAST 무릎점이 시간만(18.78%)과 같아지고 약 12 GB면 절감이 0이 된다. 작업당 1 kW 회계에서 작업이 긴 Azure의 손익분기는 약 60 GB다. 이 가정에 매이지 않게 옮긴 작업마다 같은 GB를 보낸다고 두고 적으면, 시간만과 같아지는 전송량은 옮긴 작업의 계산 에너지 1 kWh당 합성 12.9 GB, Azure 12.0 GB이며, 작업 전력이 1 kW보다 작으면(Azure에서 코어 수로 전력을 줄이면) 작업당 손익분기도 같은 비율로 준다. VM의 실제 전력(대부분 2~4코어)으로 두면 손익분기는 VM당 1 GB 안팎으로 줄어, VM을 옮기는 순이득은 디스크 이미지 크기에 따라 부호가 바뀔 수도 있다(추정). 0.06 kWh/GB는 2015년 값이라 지금 값으로는 손익분기가 더 크다. 건수로는 옮긴 작업의 55.2%가 1 GB 전송 에너지가 계산 에너지보다 큰 짧은 작업이지만, 이들의 배출은 옮긴 작업 배출의 0.69%다. 목적식에 이동 비용이 없는 것은 한계다.

4.4 스케줄러 평가

스케줄러만 쓴 시간만은 홈 리전을 떠나지 않고 시각만 옮겨 18.78%를 줄인다. 평균 2.17시간 미루며, 최대 동시 실행은 상한 17에 닿되 넘지 않는다. 스케줄러가 용량을 어떻게 다루는지는 캘리포니아의 한 날을 열어 보면 분명하다. 그림 6은 용량 무시 실행에서 캘리포니아의 동시 실행이 연중 가장 높았던 날인 7월 7일의 탄소 곡선, 그날 시작해 미뤄진 작업 가운데 창이 가장 넓은 9건, CAST의 시각별 동시 실행을 위아래로 겹친 것이다. 그날 CAST의 동시 실행은 하루 24시간 가운데 8시간 동안 상한 17에 닿아 있고, 그 시간은 현지 9시에서 16시, 곧 태양광이 만드는 한낮의 골이다.

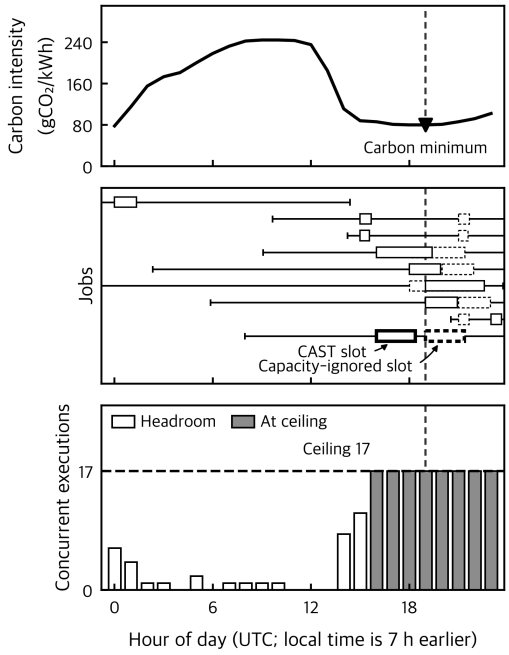


그림 6 용량이 저탄소 시각으로의 몰림을 막는 방식. 캘리포니아의 한 날(7월 7일)이며 그날 시작해 미뤄진 작업 중 창이 19.48시간 이상인 9건이다. 가는 선이 창, 실선 상자가 CAST의 실행 구간, 파선 상자가 용량 무시에서 받은 시각이며(맨 위 행은 전날 19시 UTC라 그림 밖), 굵은 테두리(맨 아래 행)가 본문의 예다. 아래 단은 CAST의 동시 실행과 상한 17이다.

Fig. 6 How capacity keeps jobs from piling into the low-carbon hours. One day in California (7 July), showing the 9 deferred jobs started that day with windows of at least 19.48 h. Thin lines are windows, solid boxes CAST's executed intervals and dashed boxes the slots received under capacity ignored (off the frame for the top row, at 19:00 UTC the previous day), with the bold outlines (bottom row) marking the example in the text; the bottom panel shows CAST's concurrency against the ceiling of 17.

작업들이 가고 싶어 하는 시각은 모두 같은 골이며, 용량을 보지 않았다면 받았을 시각(파선 상자)과 CAST가 실제로 준 시각(실선 상자)이 어긋나는 것은 그 골이 이미 상한까지 찼기 때문이다. 굵은 테두리로 표시한 작업은 현지 0시 58분에 제출된 2.36시간짜리 작업이다. 용량을 보지 않았다면 그날 탄소가 가장 낮은 시각(현지 11시와 12시가 같다) 가운데 12시에 시작했겠지만, 그 시각이 이미 상한에 닿아 있어 CAST는 이 작업을 원하던 자리보다 세 시간 앞당긴 현지 9시에 시작했다.

용량을 언제, 무엇과 함께 확인하느냐가 결과를 정한다. CAST와 같은 선형 계획, 예측, 무릎점 규칙, 풀이기 위에서 확인 방식만 다른 대조군(3.4절)을 견주었다. 도착 슬롯에 리전과 시작 시각을 확정하되 미래 슬롯의 용량을 이미 확정한 모든 계획과 함께 보는 도착 고정형은 LSTM, 완전예지, Azure에서 모두 위반이 0이다. 남의 계획을 보지 않고 확정하면서 시작 때의 확인마저 뺀 시작 확인형은 채점 작업의 9.0~17.0%(시드 포함, Azure 6.6~9.1%)가 상한을 넘긴 순간에 시작한다. 시작 때 다시 확인하는 입장 통제형에서는 위반이 LSTM 227건, 완전예지 387건(0.16~0.27%, 시드 포함 0.16~0.56%, Azure 1.3%)으로 줄지만, 이 위반은 모두 여유가 다한 작업을 처음 정한 리전에 그대로 밀어 넣는 규칙에서 나왔고 그 순간 다른 리전에는 늘 빈자리가 있었다. 거절된 작업이 리전과 시각을 다시 고르게 해도 이 규칙이 남으면 위반이 160~299건(Azure 608건) 생긴다. 꽉 찬 리전의 작업을 그 순간 자리가 있는 리전으로 보내는 대피형은 예측을 섞은 다섯 판, 24시간 전 값, Azure의 두 예측기 모두에서 위반이 0이다. 예측이 정확할수록 미룸이 느는 것(대피형 2.06시간에서 2.48시간)은 이 설계만의 일이 아니라 CAST(1.47시간에서 2.13시간)와 도착 고정형(2.05시간에서 2.49시간)에서도 같아 몰림의 결과로 볼 수 없으며, 정확한 예측이 더 깊은 저탄소 구간을 보여 주기 때문으로 보인다.

매 슬롯 다시 풀며 지금 시작만 확정하면 미래 슬롯 행 없이도 위반이 없다. 순간 확인 롤링은 LSTM, 완전예지, 24시간 전 값, Azure에서 모두 위반 0이다. 위반 0은 지금 시작만 확정하고 지금 슬롯을 순간 행으로 세는 구성과 집계 여유(이용률 합성 8.4%, Azure 26.1%)에서 따라 나오며, 이용률이 가장 높은 조건(합성 상한 7의 20.5%, Azure 상한 0.75배의 43.2%)까지 위반은 0이었다. 미래 슬롯 행이 하는 일은 위반을 막는 것이 아니라 붐빌 슬롯을 미리 피해 절감을 조금 더하는 것으로, 예약 없는 CAST가 순간 확인 롤링보다 LSTM에서 0.67, 완전예지에서 0.72, 24시간 전 값에서 0.44, Azure에서 0.12%포인트를 더 줄인다(지연을 맞춘 짝은 아니다). Azure 대표 설정에서 순간 확인 롤링(51.22%)이 CAST(46.14%)보다 높은 것은 식 (12)가 없어 예약 비용이 없기 때문이며, 예약 없는 CAST(51.35%)와는 0.12%포인트 차다.

도착 고정형과 CAST는 둘 다 위반이 없다. 위반 0에 이르는 길은 이미 정한 배정을 모두 보는 것[9]과 확정하지 않고 매 슬롯 다시 푸는 것 둘이고, 다시 푸는 것의 몫은 절감에 있다. 두 설계를 작업당 평균 지연이 같은 고정 α로 견주면, 합성 워크로드에서는

CAST(α=0.5375, 지연 약 41 ms, 68.56%)가 도착 고정형(α=0.5437, 66.92%)보다 기준 배출의 1.64%포인트를 덜 배출한다(95% 신뢰구간 1.10~2.25). 다시 뽑은 네 벌(지연 약 48 ms)에서도 1.75~2.47%포인트로 모두 0 밖이고, 완전예지에서도 2.05%포인트가 남으므로 이 몫은 새 예측이 아니라 새로 도착한 작업과 함께 다시 짜는 데서 온다. 실행 시간 가중 지연으로 짝을 지어도 CAST가 1.19%포인트(신뢰구간 0.63~1.81) 덜 배출하고, 미룸도 CAST가 짧다(1.67~2.31시간 대 2.09~2.58시간). 예약 ρ=13을 둔 대표 설정의 Azure에서는 방향이 반대여서 도착 고정형이 같은 작업당 지연에서 3.62%포인트(신뢰구간 2.65~4.75), 다시 뽑은 두 벌에서 3.72와 5.71%포인트, 실행 시간 가중 지연 짝에서 2.10%포인트(1.01~3.35) 더 줄이고, 무릎점끼리도 앞선다(48.67% 대 46.14%). 이 역전은 예약에서 온다. 예약 행(식 (12))이 깎는 절감이 CAST에서는 5.20%포인트, 도착 고정형에서는 0.37%포인트이고, Azure의 ρ=13은 넘어온 긴 VM을 두 번 센 과대값이다(3.3절). 새로 도착하는 몫에 맞춘 ρ=6에서는 CAST가 50.44%(41.28 ms)로 도착 고정형(48.67%, 42.56 ms)보다 지연을 덜 쓰고도 앞서고(예약 없는 도착 고정형 49.04%보다도), 위반 0에는 예약이 필요 없다. 예약을 빼면 무릎점끼리 CAST가 51.35%(36.95 ms)로 도착 고정형 49.04%(42.39 ms)보다 지연을 5.4 ms 덜 쓰고도 2.31%포인트 더 줄인다(미룸은 2.25시간 대 1.84시간). 예약 없이 같은 지연의 고정 α로 짝을 지어도 CAST가 작업당 지연 짝(α=0.5375 대 0.5250)에서 3.91%포인트(신뢰구간 2.90~4.93), 실행 시간 가중 지연 짝에서 2.53%포인트(1.68~3.42) 덜 배출하며, 미룸은 CAST가 2.51시간으로 도착 고정형(1.80~1.81시간)보다 길다. 남의 계획을 보지 않고 확정하되 시작 때 빈 리전으로 보내는 대피형과 같은 고정 α(0.5375)로 견주면, CAST(68.56%)가 대피형(67.88%, 작업당 지연 41.07 ms)보다 기준 배출의 0.68%포인트(신뢰구간 0.46~0.91)를 덜 배출하고 미룸도 짧다(1.67 대 2.08시간). 실제 운용 규칙인 무릎점의 CAST(71.93%)는 같은 실행 시간 가중 지연의 도착 고정형 고정 α=0.5862(72.08%)와 배출이 같고(차 0.16%포인트, 신뢰구간이 0을 포함) 미룸이 31% 짧다(1.47 대 2.13시간). CAST의 이점은 대부분 매 슬롯 다시 풀며 지금 시작만 확정하는 데서 오며, 미래 슬롯 행은 0.12~0.72%포인트를 더하고 초과 벌점과 예약은 이 워크로드에서 몫이 없다. 표 3의 무릎점끼리의 차는 지연이 달라 같은 조건이 아니다.

별도로 구현한 2단계 설계(3.4절)는 시작 때 내부 상한으로 들이는 입장 통제의 한 꼴이며 방향이 같다. 내부 상한을 물리 상한 17과 같게 두면 위반이 LSTM 18건에서 완전예지 152건으로 늘고(표 3), 같은 LSTM 예측에 완전예지를 섞어 옮기면 대체로 늘어 최대 164건에 이른다. 이 설계는 확인 시점 말고도 리전 단계, 시각 점수, 예약과 재풀이가 없다는 점, 풀이기가 달라 교란 요인이 많다. 내부 상한을 0.8배로 두면 위반이 0~19건으로 대부분 흡수되고(표 3), 24시간 전 값, 하루 한 번 발행한 LSTM, 잡음을 더한 완전예지 사이의 순서는 한 방향이 아니다.

용량을 모든 대기 작업이 함께 보는 제약으로 두면 이 증가가 없다. 리전을 도착 때 고정하는 분리형과 CAST 모두, 무릎점과 α=1 각각에서 예측기를 세 가지로 바꾼 열두 실행이 전부 위반 0이다. 예약을 0으로 두어도 두 설계와 세 예측기의 여섯 조합 모두 위반이 0이고, 절감은 예약 3일 때와 0.3%포인트 안에서 같다. Azure 워크로드에서도 예약 없이 위반이 0이다(4.7절). 위반이 없는 것은 예약 때문이 아니다. 다만 이 결과는 슬롯 안에 도착하는 작업을 슬롯 머리에서 모두 안다는 가정 아래의 것이며, 특히 미룰 수 없는 작업이 이 선견의 덕을 본다. 그래서 이 선견을 없앤 판(미룰 수 있는 작업은 다음 슬롯 머리에, 미룰 수 없는 작업은 도착 순간 하나씩 배정)을 합성과 Azure, 예약 유무, 예측기를 바꾸어 다섯 번 돌렸고, 모두 위반이 0이었으며 절감은 0.56~1.62%포인트 줄었다. 리전을 먼저 정하는 분리형도 미룬 작업이 그날 탄소 하위 세 시각에 시작하는 몫이 LSTM 23.9%에서 완전예지 52.5%로 늘지만, 여덟 예측기 변형(24시간 전 값, LSTM, 완전예지, 하루 한 번 발행한 LSTM, 잡음을 더한 완전예지, 완전예지를 25, 50, 75% 섞은 LSTM) 전부에서 위반이 0이다. 탄소만 보는 α=1에서는 두 설계 모두 완전예지가 가장 높은 절감을 내고, 세 예측기의 차이도 0.6%포인트 안이다.

용량을 보지 않을 때와 CAST의 차이를 한 주로 늘려 보면 그림 7과 같다. 7월 7일이 든 주(day 184~190)에서 용량 무시는 한낮마다 상한을 넘겨 최대 63건에 이르고, CAST는 같은 시각을 17건에서 멈춘다.

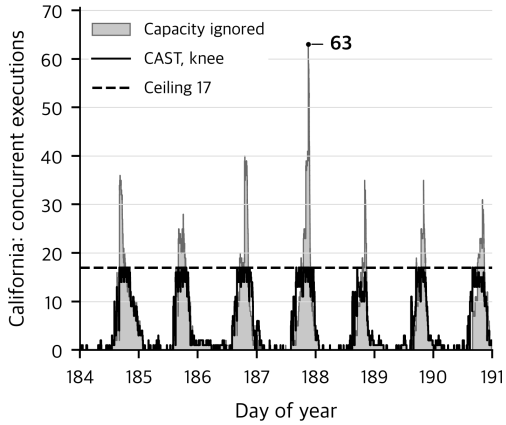


그림 7 캘리포니아의 동시 실행, 용량 무시와 CAST. 연중 한 주(day 184~190, 1월 1일을 0으로 센 날로 7월 4~10일)이며 회색 채움이 용량 무시, 검은 선이 CAST, 파선이 상한 17이다.

Fig. 7 California concurrent executions under capacity ignored and CAST. One week of the year (days 184–190 counted from 0 on 1 January, i.e. 4–10 July); gray fill is capacity ignored, the black line CAST, and the dashed line the ceiling of 17.

분리형에서 용량 행을 지우면 위반이 13,845건, 초과 시간이 599시간 생기고 최대 동시 실행이 45건에 이르지만, 절감은 48.42%로 용량이 있을 때의 48.88%보다 오히려 조금 낮다. 상한이 없으면 무릎점 규칙의 후보 열한 개가 모두 비교 대상에 들어가 정규화가 바뀌고, 무릎점이 α 평균 0.438에서 0.434로, 평균 지연이 23.4 ms에서 22.9 ms로 지연 쪽으로 옮겨 간다. 골에 몰린 작업이 더 깨끗한 시각을 얻는 것도 아니다. 분리형에서 용량 행은 절감을 깎지 않고 위반만 없앤다.

작업 143,200건은 모든 방식에서 전량 마감 이내에 처리되었다. 후보 시각을 마감 이내에서만 찾으므로 설계상의 결과다.

4.5 결합과 무릎점

로드밸런서와 스케줄러를 한 문제로 합친 값은 가중치에 따라 다르다. 같은 규칙으로 고른 무릎점끼리 비교하면 CAST는 공간만보다 10.35%포인트를 더 줄이고, α=1에서는 1.70%포인트를 더 줄인다(표 4). α=1에서는 작업 대부분이 가장 깨끗한 리전으로 가므로 시각을 옮겨 더 얻을 것이 적다. 다만 두 무릎점은 고른 α의 평균이 공간만 0.488, CAST 0.556으로 달라서 같은 α의 비교가 아니다. 분리형의 무릎점이 공간만보다 낮은 것(48.88% 대 61.57%)도 시간 축을 더해서가 아니라 무릎점이 고른 자리 때문이다.

표 5 같은 고정 α에서 CAST와 분리형의 비교. 칸은 절감률(%)이며 두 실행의 작업당 평균 지연 차는 α=0.6의 1.00 ms를 빼면 0.54 ms 안이다. 차는 분리형 배출에서 CAST 배출을 뺀 값(양수 = CAST가 덜 배출, kg)이며 괄호는 7일 단위 블록 부트스트랩의 95% 신뢰구간, pp는 기준 배출 대비 %포인트다.

Table 5 CAST versus the decoupled design at the same fixed α. Cells are savings (%); the two runs differ in per-job mean latency by at most 0.54 ms except 1.00 ms at α=0.6. Diff. is decoupled minus CAST emissions in kg (positive = CAST emits less) with 95% 7-day block-bootstrap intervals; pp is the gap in percentage points of baseline emissions.

α	CAST	Decoupled	Diff. (95% CI)	pp
0.4	38.64	37.80	238 (66 to 434)	0.83
0.5	62.99	62.02	276 (205 to 351)	0.96
0.6	77.69	76.15	441 (354 to 528)	1.54
0.7	88.99	88.29	200 (114 to 282)	0.70
0.8	93.51	93.31	59 (9 to 109)	0.20
0.9	95.24	94.98	73 (34 to 115)	0.25
1.0	95.65	95.34	91 (52 to 135)	0.32

설계끼리는 같은 α에서 비교해야 한다. 상한이 기준 피크의 1.25배인 이 조건에서 두 설계의 고정 α 곡선은 거의 겹친다. 그러나 같은 α끼리 비교하면 CAST가 α=0.4부터 1까지 일곱 점에서 0.20~1.54%포인트 덜 배출하고 신뢰구간이 0을 포함하지 않는다(표 5). 다만 지연을 맞춘 짝은 아니어서 α=0.6에서는 CAST가 실행 시간 가중 지연을 1.71 ms 더 쓰며, 지연 차를 보정하면 차는 0.35~0.6%포인트이고 α=0.6의 봉우리는 지연 차에서 온다. $\alpha\leq$0.3에서는 작업당 지연이 3 ms 아래이고 부호가 섞인다(0.3에서 분리형이 1.11%포인트 앞선다). 다만 이 짝 차가 워크로드 시드에 따라 얼마나 흔들리는지는 재지 않았으므로, 결합형은 근소하게 더 줄인다고만 적는다. 이 차는 합성 워크로드의 결과이며, 시간 이동이 거의 절감을 내지 못하는 Azure 워크로드에서는 성립하지 않는다(4.7절). 차이는 α=0.6에서 가장 크고 α=0.8 이상에서는 0.2~0.32%포인트로 작아진다.

선형 계획이 얼마를 더하는지는 탐욕과 비교하면 드러난다. 탐욕은 무릎점에서 71.08%를 내며 위반이 없고 최대 동시 실행도 17이다. 작업당 평균 지연이 0.29 ms 안으로 같은(짝을 맞춘 것은 아니며 실행 시간 가중 지연은 0.73 ms 다른) CAST는 탐욕보다 242 kg, 기준 배출의 0.84%포인트를 덜 배출한다(95% 신뢰구간 160~322 kg).

무릎점이 고정 α보다 나은지는 한 축으로 물을 수 없다. α=1이 탄소의 최솟값이고 α=0이 지연의

표 6 무릎점과 같은 지연의 고정 α 짝(CAST). 짝은 목적식과 같은 실행 시간 가중 평균 지연이 무릎점과 약 0.5 ms 안에서 같은 고정 α이고, 지연은 무릎점 / 짝의 실행 시간 가중 평균 지연, Δ는 무릎점이 짝보다 더 배출한 양(괄호는 각 워크로드 기준 배출 대비 %포인트), 신뢰구간은 7일 블록 부트스트랩 2,000회의 95% 구간이다. 마지막 열은 작업당 평균 지연으로 맞춘 짝과의 차(%포인트)다. 위 세 행은 합성 워크로드의 상한 17, 다음 두 행은 LSTM에서 상한을 10과 7로 줄인 것, 그다음 네 행은 합성 워크로드를 시드 1~4로 다시 뽑은 것, 마지막 행은 Azure 워크로드(상한 43)다.

Table 6 Knee point versus the fixed α at the same latency (CAST). The pair is the fixed α whose duration-weighted mean latency, the measure used by the objective, is within 0.5 ms of the knee's; Lat. gives knee / pair duration-weighted latency, Δ is the knee's extra emission over the pair (percentage points of each workload's baseline in parentheses), and the CI is the 95% interval of 2,000 seven-day block bootstrap draws. The last column is the gap to the pair matched on per-job mean latency (percentage points). The top three rows use the synthetic workload at the ceiling of 17, the next two LSTM with the ceiling cut to 10 and 7, the next four the synthetic workload re-drawn with seeds 1–4, and the last row the Azure workload (ceiling 43).

Condition	Knee (%)	Lat. (ms)	Pair α	Δ kg (pp)	95% CI (kg)	Per-job Δ (pp)
LSTM	71.93	47.48 / 47.08	0.5750	534 (1.87)	31 to 1,072	0.93
SN-24h	64.44	36.74 / 36.69	0.5281	537 (1.88)	12 to 1,117	3.51
Oracle	67.20	38.83 / 38.48	0.5375	628 (2.20)	96 to 1,157	3.25
LSTM, cap 10	66.11	41.00 / 41.26	0.5540	756 (2.64)	336 to 1,191	2.64
LSTM, cap 7	56.79	30.63 / 30.15	0.5250	235 (0.82)	−9 to 484	2.19
LSTM, seed 1	71.39	46.86 / 47.02	0.5750	650 (2.28)	154 to 1,163	1.41
LSTM, seed 2	70.92	46.00 / 45.79	0.5750	555 (1.94)	118 to 999	1.04
LSTM, seed 3	71.95	47.36 / 46.86	0.5750	509 (1.77)	2 to 1,042	0.87
LSTM, seed 4	71.37	46.22 / 46.26	0.5750	584 (2.04)	132 to 1,054	1.19
Azure	46.14	39.78 / 39.70	0.5563	420 (0.20)	−2,389 to 3,491	−1.89

최솟값이기 때문이다. 고정 α 열한 개를 각각 1년 동안 돌려 얻은 (평균 지연, 절감)의 점이 그림 8에 있다. 열한 점을 이은 선은 보간일 뿐이라 그림의 보조선으로만 두고 판정에는 쓰지 않는다. 판정은 같은 지연 짝으로 한다(4.1절).

분리형에서는 무릎점이 같은 지연의 고정 α에 뒤진다. 무릎점의 평균 지연은 작업당 23.4 ms, 실행 시간 가중 20.45 ms다. 작업당 지연으로 맞춘 짝인 고정 α=0.4375는 22.95 ms에서 55.49%를 내어, 무릎점(48.88%)이 이 짝보다 1,894 kg, 기준 배출의 6.62%포인트를 더 배출한다(95% 신뢰구간 1,543~2,256 kg 더). 무릎점 규칙이 쓰는 실행 시간 가중 지연으로 맞춘 짝(α=0.4156, 20.57 ms)에서도 무릎점이 666 kg(2.33%포인트) 더 배출해(188~1,141 kg 더) 결론은 같다. 분리형의 무릎점은 α를 너무 낮게 골라 작업을 홈에 남긴다. 분리형 무릎점이 예측기에 따라 CAST와 다른 방향으로 움직이는 까닭(표 3)은 가르지 않았다.

CAST에서는 무릎점 규칙이 쓰는 실행 시간 가중 지연(47.48 ms)으로 맞춘 짝(α=0.575, 47.08 ms, 73.79%)보다 무릎점이 534 kg, 기준 배출의 1.87%포인트를 더 배출한다(95% 신뢰구간 31~1,072 kg 더). 작업당 지연(47.85 ms)으로 맞춘 짝(α=0.5687, 47.71 ms, 72.86%)과는 268 kg(0.93%포인트)으로 신뢰구간이 0을 포함한다(252 kg 덜 ~ 808 kg 더). 0.5687과 0.575는 1년을 돌려 찾은 값이지만, 아래에서 보듯 첫 주만 보고 골라도 비슷한 자리에 온다. 무릎점의 평균 미룸은 1.47시간으로 작업당 짝의 1.63시간보다 짧고, 95 백분위수도 9.98시간 대 10.95시간이다. 미룸은 목적에 없는 지표이고 시간 이동을 덜 한 것의 다른 면일 수 있으므로, 이 차는 장점이 아니라 사실로만 적는다. 같은 규칙이 분리형에서는 6.62%포인트 뒤진다.

실행 시간 가중 짝을 합성 워크로드의 아홉 조건에서 찾으면(표 6), 무릎점은 여덟 조건에서 1.77~2.64%포인트 더 배출하고 상한 7에서만 차이가 없다(0.82%포인트, 신뢰구간 −9~484 kg). 대표 조건과 시드 네 벌을 묶으면 평균 1.98%포인트(시드 사이 표준편차 0.19)이고, 작업당 지연 짝으로 재도 평균 1.09%포인트로 0 밖이다. Azure에서는 실행 시간 가중 짝과 차이가 없다(0.20%포인트, 신뢰구간 −2,389~3,491 kg). 작업당 평균 지연 짝으로 재면 Azure에서는 방향이 반대여서, 무릎점이 같은 지연의 짝(α=0.5438)보다 Azure 기준 배출의 1.89%포인트를 덜 배출하고(95% 신뢰구간 820~6,939 kg 덜), Azure를 다시 뽑은 두 벌에서도 1.56%포인트와 1.34%포인트 덜 배출해 세 벌 모두 신뢰구간이 0 밖이다. 평균 미룸은 Azure 세 벌을 포함한 모든 조건에서 무릎점이 짧다. 무릎점이 고르는 자리는 예측 곡선의 모양에 따라 움직인다는 아래의 결과와 같은 방향이다.

고정 α 하나를 예측기와 무관하게 쓰면 어떻게 되는지도 보았다. LSTM에서 찾은 짝 α=0.5687을 다른 두 예측기에 그대로 주면 24시간 전 값에서 72.58%(46.4 ms), 완전예지에서 73.97%(46.0 ms)를 낸다. 고정 α는

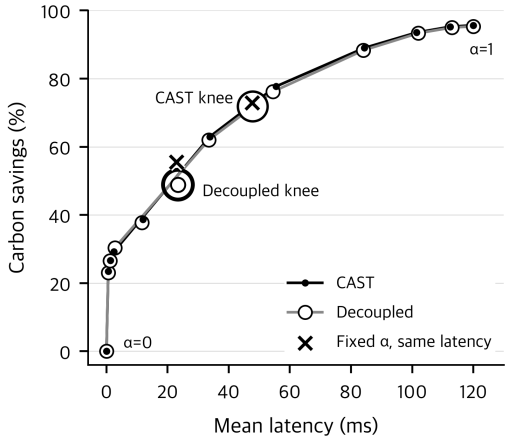


그림 8 고정 α 곡선과 슬롯별 무릎점. ●와 검은 선은 CAST, ○와 회색 선은 분리형의 고정 α 열한 점(0, 0.1, …, 1)이며, 선은 점을 이은 보조선이고 과녁 표식이 각 설계의 무릎점이고(CAST의 ×는 무릎점과 거의 같은 자리라 과녁 안에 있다), ×는 무릎점과 같은 작업당 평균 지연을 내는 고정 α(CAST 0.5687, 분리형 0.4375)다.

Fig. 8 Fixed-α curves and per-slot knee points. Filled dots with the black line are CAST and hollow dots with the gray line the decoupled design at eleven fixed α (0, 0.1, …, 1); lines only connect the points, target markers are each design's knee point (CAST's cross lies inside its target), and crosses are the fixed α giving the same per-job mean latency as each knee (0.5687 for CAST, 0.4375 for decoupled).

예측기를 바꿔도 지연 46~48 ms, 절감 72.6~74.0%에서 안정하다.

평가 구간의 첫 주(1월 8~14일)만 보고 무릎점과 같은 지연이 나오도록 고른 고정 α=0.5625도 연간 71.92%(46.27 ms)로, 무릎점과 절감이 같고 지연은 1.6 ms 짧다. 같은 지연의 고정 α는 1년을 다 보아야만 찾을 수 있는 값이 아니다. 다만 첫 주의 예측 입력에는 준비 구간의 되풀이 자료가 섞여 있다. 그래도 무릎점을 기본 운용 규칙으로 두는 것은 운영자가 가중치를 정하거나 첫 주를 돌려 맞추지 않아도 슬롯마다 탄소와 지연 전선의 한가운데를 고르기 때문이며, 같은 지연의 고정 α보다 1.77~2.64%포인트를 더 배출하는 것이 그 값이다. 합성 워크로드에서 무릎점이 같은 지연의 고정 α보다 낮지 않은 까닭의 하나는 식 (8)에 있는 것으로 보인다. 탄소 항을 슬롯마다 다른 분모로 나누므로 같은 α라도 슬롯마다 kg과 ms의 교환비가 다르고, 슬롯별 무릎점은 여기에 슬롯마다 다른 α까지 더해 교환비가 서로 다른 점들을 모은다. 같은 이유로 고정 α 열한 점을 이은 곡선도 연간 파레토 전선은 아니다.

α는 슬롯마다 하나를 골라 그 슬롯의 모든 리전과 작업에 같이 쓴다. 슬롯이 고른 α는 평균 0.556이고, 채점 구간 8,592 슬롯 가운데 0.6이 48.3%, 0.5가 40.7%, 0.7이 6.3%로 세 값이 95.3%를 차지한다.

이 분포는 예측 곡선의 모양에 따라 움직인다. 완전예지에서는 0.5를 고른 슬롯이 늘고, 아시아의 낮인 UTC 3~10시에 평균 α가 0.53에서 0.47~0.52로 가장 많이 내려간다. 완전예지는 골이 뾰족해 탄소 축의 정규화가 달라지고, 무릎점이 지연 쪽으로 반 칸 물러난다.

4.2절에서 무릎점의 순서가 뒤집힌 까닭이 이것이다. 완전예지 무릎점(67.20%)이 LSTM 무릎점(71.93%)보다 4.72%포인트 낮은 차이를 가르기 위해, 한 실행이 슬롯마다 고른 α를 다른 예측기의 실행에 그대로 주었다. LSTM 예측에 완전예지의 α 경로를 주면 69.23%(43.8 ms), 완전예지 예측에 LSTM의 α 경로를 주면 71.33%(45.2 ms)다. 두 순서를 평균하면 차이의 약 72%가 α 경로에서, 약 28%가 예측에서 온다. 상한을 없앤 실행에서도 완전예지 무릎점은 73.39%에서 69.25%로 내려가므로, 이 차이는 용량이 아니라 무릎점이 고르는 자리에서 온다.

몰림 자체는 공유 제약에서도 남는다. 완전예지 예측에 LSTM의 α 경로를 준 실행은 탄소 최저 시각의 동시 실행이 평균 6.67건으로 LSTM 실행의 4.68건보다 많고, 그 시각에 상한에 닿은 리전-일도 23.92%로 11.31%의 두 배다. 그런데 절감은 71.33%로 LSTM과 0.59%포인트밖에 차이 나지 않고 위반은 0이다. 몰린 작업은 골을 상한까지 채우는 데서 멈추고 나머지는 다음으로 깨끗한 자리로 간다. 공유 제약은 몰림을 지우지 않고 해롭지 않게 만든다.

온라인 구조가 얼마를 내주는지는 한 주를 한 번에 푼 하한과 비교해 잰다. 계절이 다른 네 주(2월 중순, 4월 하순, 7월 초, 10월 중순)마다 그 주에 제출된 작업을 상한 17 아래에서 완전예지 실측으로 한 문제로 풀었다. 시작 후보는 마감까지 전부 열고, 용량은 슬롯을 통째로 덮는 실행만 세며 그 주 이전의 점유는 넣지 않으므로 같은 격자 위의 어떤 배정보다도 배출이 낮다. 지연은 연간 실행에서 그 주 작업이 쓴 실행 시간 가중 지연을 예산으로 두어, 같은 지연에서 배출만 최소화한다. 하한과의 격차는 CAST 무릎점이 완전예지로 1.87~2.77%포인트, LSTM으로 3.11~6.49%포인트이고, 탄소만 보는 α=1에서는 0.27~1.14%포인트와 0.33~1.47%포인트다(각 주 기준 배출 대비). 같은 네 주에서 고정 α 짝(LSTM α=0.5750, 완전예지 α=0.5375)의 격차는 완전예지로 0.13~0.64%포인트, LSTM으로 1.19~2.36%포인트다. 따라서 다시 풀기와 확정

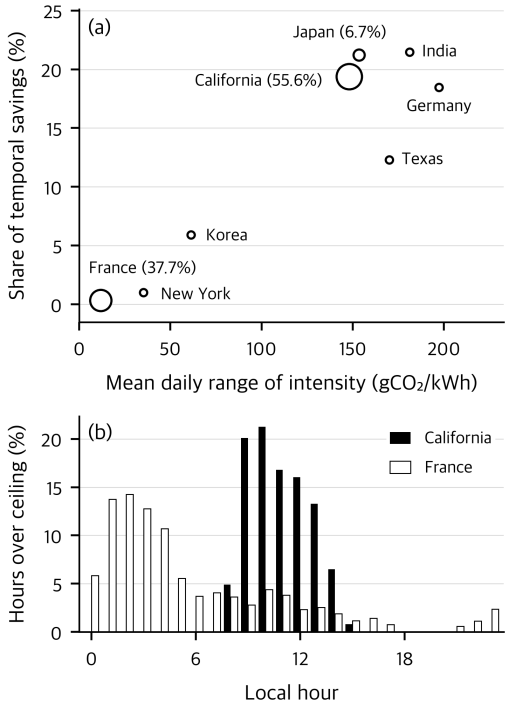


그림 9 상한이 걸리는 자리와 시간 이동의 절감이 큰 자리. (a) 여덟 리전의 하루 절대 변동(가로, 채점 구간의 일별 최대−최소 평균)과 시간 이동 절감 몫(세로, 시간만의 출발 리전별 감소량)이며, 원 넓이는 용량 무시(무릎점)에서 상한 17을 넘긴 시간의 몫에 따라 커지며 괄호 안이 그 몫이고, 작은 ○는 넘긴 시간이 거의 없는 리전이다. (b) 같은 실행에서 상한을 넘긴 시각의 현지 시간(일광절약시간 반영) 분포로, 리전별 합을 100%로 두었고 채운 막대가 캘리포니아, 빈 막대가 프랑스이며 0.5% 미만 막대는 생략했다.

Fig. 9 Where capacity binds and where temporal shifting pays. (a) Daily absolute swing (x; mean daily max−min over the scored window) against share of temporal-shift savings (y; temporal-only reduction by origin) for the eight regions; circle area grows with the share of hours over the ceiling of 17 under capacity ignored (knee α), given in parentheses, and small circles mark regions that barely exceed it. (b) Local-hour distribution of those hours (daylight saving applied), normalised per region; California filled, France hollow, with bars below 0.5% omitted.

방식이라는 온라인 구조의 대가는 작고(합성 작업의 시작 창이 모두 24시간 안이라 지평의 대가는 이 비교로 잴 수 없다), 무릎점의 격차는 대부분 무릎점 규칙에서, LSTM 쪽의 더 큰 격차는 예측 오차에서 온다. 하한은 통째로 덮는 실행만 세는 완화라 실제 최적보다 낮을 수 있어, 이 격차는 크게 잰 값이다.

4.6 상한이 걸리는 자리

평균 배출계수로 셀 때 상한이 어느 리전에서 걸리는지는 용량 무시 실행에서 상한을 넘긴 시간을 리전별로 나누어 보면 드러난다. 상한이 모든 리전에 같으면 작업이 모이는 리전에서 넘치는 것은 거의 정해진 일이고, 실제로 상한을 넘긴 시간의 55.6%가 캘리포니아, 37.7%가 프랑스, 6.7%가 일본에서 나며 나머지 리전은 거의 넘기지 않는다(인도 0.02시간). 캘리포니아와 프랑스 두 리전이 93.3%다. 리전마다 상한을 달리 두어도 이 두 리전이 앞에 서는지는 4.7절에서 본다.

이 자리를 시간 이동의 절감이 큰 자리와 견주어 본다(그림 9(a)). 홈을 고정하고 시각만 옮긴 시간만의 절감 5,377 kg을 출발 리전별로 나누면 인도 21.5%, 일본 21.2%, 캘리포니아 19.4%, 독일 18.5%, 텍사스 12.3%다. 인도와 일본은 자료 산정 방식이 바뀐 것으로 보이는 리전이라(4.2절) 이 순위는 자료에 기댄다. 이 다섯은 하루 안의 절대 변동, 곧 날마다 최댓값과 최솟값의 차를 평균한 값이 148.1~197.4 gCO_2/kWh로 가장 큰 다섯 리전이고, 시간 이동 절감의 92.8%를 낸다. 작업당 1 kW로 절대 배출량을 세는 회계의 정의에서 따라 나오는 비례다. 캘리포니아는 날마다 최댓값이 최솟값의 평균 3.93배다. 프랑스는 날마다 (최댓값−최솟값)/일평균을 1년 평균한 상대 변동이 0.97로 캘리포니아(1.06) 다음으로 크지만, 연평균이 13.6 gCO_2/kWh로 낮아 절대 변동이 11.8로 가장 작고 시간 이동 절감의 0.3%만 낸다.

두 목록에 함께 드는 리전은 캘리포니아와 일본이다. 캘리포니아는 CAST에서 실행된 몫의 27.3%를 받는 목적지이면서 시간 이동 절감의 19.4%를 내는 출발지이고, 용량을 두지 않으면 초과 시간의 55.6%가 나는 리전이다. 일본은 시간 이동 절감의 21.2%와 초과 시간의 6.7%를 낸다. 균일 상한에서 상한이 공간 이동이 모이는 곳에서 걸리는 것은 상한의 정의에서, 시간 이동의 절감이 절대 변동이 큰 곳에서 나는 것은 kg 회계에서 거의 따라 나온다. 이 절의 측정이 보태는 것은 두 목록이 캘리포니아와 일본(나라 단위 구역 자료)에서 겹치고 인도, 독일, 텍사스에서는 상한이 거의 걸리지 않는다는 것, 그리고 상한 배치를 바꿔도 이 두 리전이 앞에 선다는 것(4.7절)이다.

두 리전은 넘치는 시각도 다르다(그림 9(b)). 캘리포니아는 상한을 넘긴 시간이 모두 일광절약시간을 반영한 현지 8시부터 18시 전 사이에 있고, 그 74.2%가 9시부터 13시 전까지 네 시간에 몰린다. 태양광이 만드는

한낮의 골로 작업이 함께 모인 결과다. 프랑스는 반대로 현지 밤에 넘친다. 가장 큰 시간대가 2시로 14.3%이고, 0시부터 5시 전까지가 57.3%다. 프랑스의 하루 최저는 수요가 낮은 밤에 오고, 독일·인도·뉴욕에서 온 작업이 그 시각으로 모인다.

4.7 결론이 성립하는 범위

상한을 바꾸면 결과가 상한값에 얼마나 매였는지 드러난다. 상한을 모든 리전에 같게 기준 피크 13의 0.5배(7)부터 3배(39)까지 일곱 단계로 바꾸어 CAST를 돌렸다. 예약 ρ는 미룰 수 없는 작업이 준비 구간에서 필요로 한 자리이므로 상한과 무관하게 모든 단계에서 3으로 고정했다. 일곱 단계 모두 위반과 초과 시간이 0이다. 기준 피크의 절반인 상한 7에서는 홈 리전 즉시 실행조차 713번 상한을 넘기지만, CAST는 무릎점에서 56.79%를 줄이며 한 번도 넘기지 않는다.

절감은 상한을 늘릴수록 오르다가 기준 피크의 2배 근처에서 멈춘다. 무릎점 절감은 상한 7, 10, 13, 17, 20, 26, 39에서 56.79%, 66.11%, 70.16%, 71.93%, 72.72%, 73.21%, 73.42%이고, 3배에서는 73.42%로 용량 무시(73.39%)를 0.03%포인트 넘는데, 무릎점이 상한에 따라 α를 다시 고르기 때문이며 용량 무시는 고정 α에서만 상한이다. 용량을 지키는 비용, 곧 용량 무시와의 차는 대표 상한 17에서 1.47%포인트, 기준 피크와 같은 13에서 3.24%포인트, 절반인 7에서 16.61%포인트이고, 2배인 26에서는 0.19%포인트로 거의 사라진다. α=1에서도 상한 7, 10, 13에서 76.30%, 89.71%, 94.18%, 20에서 95.80%, 39에서 95.96%로 같은 꼴이며, 여섯 판 모두 위반이 0이다.

상한이 조여도 무릎점이 고르는 α는 거의 움직이지 않는다. 슬롯이 고른 α의 평균은 상한 7에서 0.530, 17에서 0.556, 39에서 0.554다. 상한이 조이면 무릎점이 지연 쪽으로 물러서는 대신 갈 자리가 없는 작업이 홈에 남아, 홈 리전에서 도는 작업이 상한 39의 41%에서 상한 7의 50%로 는다. 리전을 도착 때 고정하는 분리형은 상한을 풀어도 무릎점 절감이 상한 7~26에서 48.44~49.71%에 머문다. 분리형의 무릎점은 α를 0.44 안팎으로 두어 작업을 홈에 남기므로, 상한이 결과를 정하지 않는다.

예약이 없어도 대표 상한 17에서는 위반이 나지 않는다. 예약 ρ를 0, 2, 3, 4, 6으로 바꾸면 절감은 72.06%, 72.16%, 71.93%, 72.00%, 71.53%로 ρ=3과 0.40%포인트 안에서 오르내리고, 위반은 모두 0이다. 매 슬롯 i=0만 확정하므로 아직 확정하지 않은 미래 슬롯의 계획은 언제든 비켜 줄 수 있고, 이미 시작해 돌고 있는 작업만으로 슬롯이 찬 때가 없었기 때문이다. 예약이 바꾸는 것은 미룰 수 있는 작업 가운데 마감까지 남은 여유가 한 슬롯 아래로 떨어져 그 자리에서 시작해야 했던 작업의 수로, ρ를 0에서 6으로 늘리면 4,805건에서 3,931건으로 준다. 상한을 조여도 마찬가지다. 상한 7과 10에서 예약을 0으로 두면 절감은 58.30%와 66.85%로 예약 3일 때보다 1.51%포인트와 0.74%포인트 높고, 위반은 여전히 0이다. 대신 마감 직전에 시작이 강제되는 작업이 상한 7에서 13,479건으로, 예약 3일 때(4,625건)의 세 배 가까이 된다. 어느 상한에서도 위반이 없는 것은 예약에 기대지 않으며, 예약은 위반을 막은 적이 없고, 마감 직전으로 밀리는 작업을 줄이는 선택 사항으로 대표 실행에 남겼다. Azure에서는 예약을 빼면 강제 시작이 5,491건에서 10,815건으로 두 배가 되지만 절감이 5.20%포인트 늘어, 이 워크로드에서 예약은 쓸모를 보이지 못했다.

같은 α에서 결합과 분리형의 차는 상한을 조일수록 벌어진다. α=1에서 CAST와 분리형의 차는 대표 상한 17에서 0.32%포인트(표 5)이고, 13에서 0.89%포인트, 10에서 1.77%포인트, 7에서 3.04%포인트다. 리전을 도착 때 고정하면 그 리전의 자리가 찼을 때 다른 리전으로 비켜 갈 수 없기 때문으로 보인다. 무릎점끼리는 두 설계가 고르는 α가 달라 같은 α의 비교가 되지 않으므로 여기서 견주지 않는다.

리전마다 상한을 달리 두어도 결론은 같다. 총합을 113~114로 거의 같게 두고 세 가지로 나누었다. 리전마다 자기 기준 피크의 1.25배를 주면 70.04%, 깨끗한 세 리전(캘리포니아·프랑스·뉴욕)을 크게 주면 73.05%, 작게 주면 63.75%이고 모두 위반이 없다. 같은 세 배치로 용량 무시의 배정을 채점하면 상한을 넘긴 시간은 자기 피크의 1.25배에서 캘리포니아 47.5%·프랑스 46.6%, '작게'에서 프랑스 57.5%·캘리포니아 40.1%, '크게'에서 캘리포니아 44.8%·프랑스 31.4%·일본 21.3%로, 어느 배치에서도 두 리전이 앞에 선다. 다만 시간만의 리전별 절감은 상한 17에서 잰 것이라, 상한 배치를 바꿀 때 그 목록이 그대로인지는 재지 않았다.

워크로드 추첨에 따른 변동은 합성 워크로드를 시드 넷으로 다시 뽑아 잰다. 시드마다 기준 피크가 달라 상한과 예약도 같은 규칙으로 다시 정했으므로(상한 14~17, 예약 2~3), 아래 폭에는 상한의 변동도 들어 있다. 주 실험을 포함한 다섯 벌에서 CAST 무릎점은 71.51 ± 0.43%, α=1은 95.40 ± 0.28%, 분리형은 49.01 ± 0.28%, 공간만은 61.20 ± 0.22%, 시간만은 18.72 ± 0.05%, 용량 무시는 73.34 ± 0.27%이고(평균 ± 표준편차), 공유 제약이나 다시 풀기를 쓰는 실행에서는 모두 위반이 0이다. 무릎점끼리의 차는 CAST와 분리형이 22.50 ± 0.66%포인트, CAST와 공간만이 10.32 ± 0.34%포인트이며 다섯 벌 모두에서 순서가 같다. 이 차의

대부분은 설계가 아니라 무릎점 규칙이 설계마다 다른 α를 고른 결과이며, 같은 α에서의 설계 차는 표 5에 있다.

예측 지평을 6, 12, 24시간으로 바꾸면 무릎점 절감은 71.63%, 71.04%, 71.93%로 0.9%포인트 안에서 움직이고 위반은 모두 0이다. 지평이 짧을수록 미룰 수 있는 범위가 좁아져 평균 미룸이 0.89, 1.25, 1.47시간으로 짧아진다.

네트워크 지연에 상한을 두면 결과가 크게 바뀐다. 식 (5)의 지연 상한을 모든 작업에 50, 100, 150 ms로 두면 무릎점 절감은 28.41%, 32.98%, 66.37%, α=1 절감은 38.09%, 46.83%, 89.85%로, 상한이 없을 때(71.93%, 95.65%)보다 낮고 위반은 모두 0이다. 50 ms와 100 ms에서는 한국, 일본, 인도에서 출발한 작업이(한국·일본은 캘리포니아까지 109~134 ms, 인도는 프랑스까지 137 ms) 가장 깨끗한 캘리포니아와 프랑스에 닿지 못해 홈에 남는 작업이 84.2%와 78.2%에 이르고, 절감이 상한이 없을 때의 절반 아래로 꺾인다.

미룰 수 있는 작업의 비율 φ를 20%에서 60%까지 바꾸어도 결론은 그대로다. 등급 안의 비율은 유지하고, 상한과 예약은 각 φ의 기준 피크에서 같은 규칙으로 다시 정했다. CAST 무릎점은 φ 20, 30, 40, 50, 60%에서 72.30%, 72.54%, 71.93%, 71.45%, 70.01%, 분리형은 47.97~49.98%이고, 두 설계 모두 모든 φ에서 위반이 없다. 미룰 수 있는 작업이 늘수록 용량 무시의 위반은 각 φ의 상한으로 세어 15,233건에서 46,994건으로 세 배가 되고, 용량을 지키는 비용은 1.10~3.09%포인트로 φ=60%에서 가장 크다. 미룰 수 있는 작업이 많을수록 같은 골로 더 몰린다.

미룰 수 있는 작업의 도착 봉우리를 현지 새벽 2시에서 한낮 13시로 옮겨도 결론은 같다. 기준 피크가 바뀌어 상한은 같은 규칙으로 23이 되며, CAST는 70.26%를 위반 없이 줄인다. 시간만은 18.78%에서 13.92%로, 용량을 지키는 비용은 1.47%포인트에서 0.31%포인트로 작아지지만, 시간 이동의 절감이 큰 출발 리전은 인도, 독일, 일본, 캘리포니아로 거의 같다.

작업 수와 리전별 발생률을 바꾸어도 결론은 같다. 리전당 하루 도착을 100건으로 두 배 늘리면 기준 피크가 18, 상한이 23으로 커지며, CAST는 무릎점에서 69.45%, α=1에서 92.58%를 위반 없이 줄인다. 용량 무시는 73.30%를 내지만 위반이 81,799건, 최대 동시 실행이 122건이며, 상한의 정수 단위가 커져도 용량을 지키는 비용은 3.85%포인트로 오히려 커진다. 미국 세 리전에 하루 80, 70, 70건, 나머지 다섯 리전에 30~40건을 주어 발생률을 달리하면(하루 합은 같다) 상한은 18이 되고, CAST는 무릎점 74.53%, α=1 95.42%를 위반 없이 내며 용량을 지키는 비용은 0.71%포인트다.

Azure 워크로드에서도 공유 제약이나 다시 풀기를 쓰는 실행(CAST, 분리형, 도착 고정형, 대피형 등)은 모두 위반 없이 돈다. 용량 행을 지운 두 실행만 위반을 내며, 결합형은 61,400건과 초과 5,911시간에 최대 동시 실행 107건으로 상한 43의 2.5배에 이르고 분리형은 30,599건이다.

결과의 모양은 합성과 다르다. 작업이 길고 미룰 수 없는 작업시간이 51%라 시간 이동으로 얻을 것이 거의 없어, 시간만은 1.53%를 줄이고 CAST의 무릎점(46.14%)도 공간만(45.71%)보다 0.43%포인트 앞설 뿐이다. 그래서 CAST와 분리형의 무릎점 차도 1.13%포인트로 작고, 트레이스를 다시 뽑은 두 시드에서는 2.02%포인트와 −0.95%포인트로 부호가 바뀐다. 합성 워크로드에서 본 결합형의 몫(표 5)은 Azure에서는 성립하지 않으며, 탄소만 보는 α=1에서는 분리형이 65.35%로 CAST(64.09%)보다 1.26%포인트 앞선다. 반대로 용량을 지키는 비용은 훨씬 크다. 탄소만 보는 α=1에서 예약 없는 CAST(67.74%, 115.1 ms)와 용량 무시(95.11%, 120.5 ms, 위반 139,527건)의 차는 27.37%포인트로, 합성의 0.34%포인트와 크게 다르다. 합성은 평균 동시 실행 11.5건이 가장 깨끗한 프랑스 한 리전의 상한 17 안에 들지만, Azure는 89.9건이 프랑스의 상한 43을 넘기 때문이다. 무릎점끼리의 차 11.35%포인트(예약 없이)는 지연이 13 ms 다른 두 실행의 차다. VM 크기를 전력에 반영해 코어 수에 비례하게 다시 채점해도(배정은 그대로) Azure의 절감은 많아야 1.1%포인트(예약 없는 판) 움직이고 비교의 부호는 모두 같다. 무릎점이 예측기에 따라 뒤집히는 일도 없어 완전예지 46.55%, LSTM 46.14%, 24시간 전 값 45.74%로 0.81%포인트 안에 있다.

상한과 예약에 대한 반응도 다르다. 상한을 기준 피크 34의 0.75배, 1배, 1.25배, 1.5배, 2배로 바꾸면 무릎점 절감은 30.03%, 38.76%, 46.14%, 52.17%, 60.22%로 2배에서도 멈추지 않고, 용량 무시까지 2.48%포인트가 남는다(그림 10). 예약 ρ를 6, 13, 19, 25로 바꾸면 절감은 50.44%, 46.14%, 42.49%, 39.88%로 준다(ρ=13은 넘어온 긴 VM을 두 번 센 과대값이다, 3.3절). Azure에서는 ρ가 상한 43의 14~58%라, 예약이 미래 슬롯의 자리를 직접 깎는다. 예약을 0으로 두어도 위반은 없고 절감은 51.35%로 ρ=13보다 5.20%포인트 높아, Azure에서 예약의 비용은 합성(0.13%포인트)보다 훨씬 크다. 도착 고정형은 예약의 영향을 0.37%포인트만 받아 ρ=13에서는 CAST보다 앞서지만(48.67% 대 46.14%), 새로 도착하는 몫에 맞춘 ρ=6에서도 CAST가 50.44%로 앞서고, 예약을 빼면 CAST가 지연을 5.4 ms 덜 쓰고도 2.31%포인트 앞선다(51.35% 대 49.04%, 4.4절).

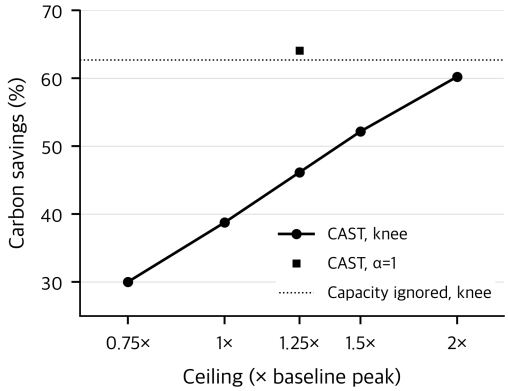


그림 10 Azure 워크로드에서 상한에 따른 절감. 가로는 상한을 기준 피크 34의 배수로 적은 것이며(로그 눈금), ●와 실선은 CAST의 무릎점, ■는 대표 상한(1.25배)에서의 CAST α=1, 검은 점선은 용량 무시(무릎점)다. 계열 이름은 표 3·표 4와 같다.

Fig. 10 Savings against the capacity ceiling on the Azure workload. The x axis is the ceiling as a multiple of the baseline peak of 34 (log scale); filled circles with the solid line are CAST at the knee, the square is CAST at α=1 at the representative ceiling (1.25×), and the black dotted line is capacity ignored at the knee. Series names follow Tables 3 and 4.

4.8 선행 정책 재현 비교

CICS, CASPER, Sukprasert, Caspian의 정책을 원문의 수식을 따르되 아래에 적은 점을 바꾸어 본 연구의 자료 위에 다시 구현하고 같은 회계(식 (1), 3.5절)로 채점하였다. CASPER는 탄소와 켜 둘 서버 대수의 가중합을 최소화하며 서버 대수를 함께 정하는 정수 계획으로, 원문 코드의 가중치 0.9와 서버 용량(서버당 10건, 리전당 100대)을 그대로 두고 매 시간 제출된 작업 수를 요청 수로 넘겼으며, 탄소는 그 시간의 실측을 쓴다. 그래서 CASPER는 본 연구의 상한 17이 아니라 스스로 정한 서버 용량(시간당 8,000건이라 사실상 묶이지 않는다) 안에서 배정하며, 시각은 옮기지 않는다. 표 7의 필요 용량은 CASPER가 고른 배정에서 나중에 잰 최대 동시 실행 수다.

CICS는 리전마다 하루 전에 선형 계획 하나를 풀어, 그날 도착한 유연 작업의 총량은 그대로 두고 시간별 시작 허용량을 탄소집약도가 낮은 시각으로 옮긴 자원 곡선을 만든다. 원문의 CPU 사용량 대신 작업 건수를 쓰고, 시간별 허용량은 하루 평균의 2배까지로 묶었다. 첨두 전력 항의 가중치는 원문에 값이 없어 0으로 두어 탄소만 보게 했고, 그날의 탄소집약도는 실측값을 미리 안다고 두었다. 원문에는 작업별 마감이 없으므로, 대기 작업을 마감이 이른 순으로 허용량만큼 내보내고 마지막 기회가 온 작업은 허용량을 넘어도 내보낸다. 리전은 옮기지 않는다.

Sukprasert는 공개 코드의 탐색을 그대로 따른다. 제출부터 마감까지의 창 안에서, 작업 길이만큼 이어진 구간의 실측 탄소집약도 합이 가장 작은 시작 시각을 1시간 간격으로 고른다. 원문처럼 1년치 탄소집약도를 미리 알고 용량은 보지 않는다. 원문 실험은 여유를 24시간 이상의 고정값으로 바꿔 가며 보지만, 여기서는 작업마다 자기 마감을 여유로 넣었다. 시간 이동은 홈 리전 위와 CAST가 정한 리전 위에서 두 번 돌리고, 공간+시간은 여덟 리전의 (리전, 시각) 짝 전체에서 같은 탐색을 한다.

Caspian은 공개 코드가 없어 논문의 정식화와 알고리즘 셋을 그대로 옮겼다. 매 1시간 24슬롯 지평의 선형 계획 완화를 풀고, 해의 값이 큰 작업부터 결합 목적의 계수가 큰 (리전, 슬롯)에 용량이 허락하는 대로 배정하며, 지금 슬롯에 시작하는 배정만 확정한다. 용량은 리전당 17의 하드 제약이고, 탄소는 LSTM 예측을 쓰며, 세 목적의 가중치는 원문 기본값대로 모두 1로, 리전별 전력 효율 계수도 1로 두었다. 원문은 실행 시간을 정수 슬롯으로 세므로 1시간 미만 작업도 한 슬롯을 통째로 차지한다. 이 부풀림을 덜려고 슬롯 안에서 쓰는 자원량을 실행 시간(최대 1)으로 둔 판을 썼다. 원문의 마감은 지키지 않아도 되는 목표라서, 마감을 1시간 넘기도록 배정받지 못한 작업은 그 시각 홈 리전에서 바로 시작하게 했다.

비교의 자는 둘이다. 하나는 상한 17에서의 위반 건수이고, 다른 하나는 그 정책의 배정이 위반 없이 돌려면 필요한 리전 상한, 곧 필요 용량이다. 상한 17은 본 연구가 고른 값이므로, 선행 정책이 그 상한을 넘겼다는 사실만으로는 정책의 좋고 나쁨을 가릴 수 없다. 필요 용량은 본 연구가 정한 상한과 무관하게 각 정책이 스스로 요구한 값이며, 1초짜리 순간 피크의 영향을 가리기 위해 시간 가중 99.9 백분위수를 함께 적는다. 결과는 표 7에, 절감과 필요 용량을 두 축에 놓은 것은 그림 11에 있다. 그림 11에는 4.7절에서 상한을 바꾼 CAST의 곡선과 합성 워크로드의 용량 무시 수준(가로 점선)도 함께 그렸다.

표 7 선행 정책을 같은 회계로 재현한 결과. 위반은 상한 17을 넘긴 배정 건수, 마감은 마감 위반 건수이고, 필요 용량은 리전별 최대 동시 실행의 최댓값(괄호는 시간 가중 99.9 백분위수), 설비 총량(Total)은 리전별 최댓값의 합이며 기준 배정의 피크는 13이다. Suk.는 Sukprasert 등[6]의 정책으로, (home)은 홈 리전 위에서, (ours)는 CAST(무릎점)가 정한 리전 위에서 시각만 옮긴 것이다. CICS는 하루 전 자원 곡선, CASPER는 탄소 인지 프로비저닝(CAP) 정책, Caspian은 슬롯 안 자원량을 실행 시간으로 둔 판이다.

Table 7 Prior policies reproduced under the same accounting. Viol. counts assignments past the ceiling of 17 and Deadl. deadline misses; Req. cap. is the largest per-region peak concurrency (time-weighted 99.9th percentile in parentheses) and Total the sum of per-region peaks; the baseline peak is 13. Suk. is the policy of Sukprasert et al. [6]: (home) shifts time on the home region and (ours) on the regions chosen by CAST at the knee. CICS uses the day-ahead capacity curve, CASPER its CAP policy, and Caspian the variant whose per-slot resource equals the run time.

Policy	Savings (%)	Viol.	Req. cap.	Total	Deadl.	Lat. (ms)
Baseline	0	0	13 (8)	89	0	0
CICS	13.89	0	12 (9)	88	0	0
Suk. temporal (home)	22.22	3,180	27 (19)	194	0	0
Suk. temporal (ours)	73.23	37,236	82 (64)	267	0	47.8
CASPER	95.30	11,061	29 (24)	46	0	120.4
Suk. spatial+temporal	96.63	61,539	129 (107)	139	0	120.4
Caspian	67.54	1,538	34 (28)	86	5,582	115.7
CAST, knee	71.93	0	17 (17)	117	0	47.8
CAST, α=1	95.65	0	17 (17)	46	0	120.0

절감률만 보면 CASPER(95.30%)와 Sukprasert 공간+시간(96.63%)이 CAST의 무릎점(71.93%)을 앞선다. 뒤엣것은 1년치 탄소집약도를 모두 미리 안다고 가정한 정책이다. 그러나 두 정책의 배치가 위반 없이 돌려면 리전 하나가 동시에 29건과 129건을 돌려야 한다. 기준 배정의 피크 13의 2.2배와 9.9배다. 지연까지 같은 짝으로 좁히면 차이가 분명하다. 둘 다 지연을 보지 않아 평균 지연이 120.4 ms와 120.0 ms로 거의 같은 CASPER와 CAST(α=1)는 절감도 95.30%와 95.65%로 비슷하지만, CAST(α=1)는 상한 17 안에서 이 절감을 낸다.

같은 축으로 좁혀 보아도 마찬가지다. 홈을 고정하고 시각만 옮기는 조건에서 CICS는 13.89%를 내며 필요 용량이 12로 기준보다도 낮다. 하루 전에 정한 자원 곡선 하나로 다음 날의 부하를 묶어 피크를 깎기 때문이며, 그 대신 개별 작업의 마감에 맞춰 시각을 고르지 않는다. Sukprasert의 시간 이동은 같은 조건에서 22.22%를 내지만 필요 용량이 27로 기준 피크의 2.1배다. 본 연구의 시간만은 18.78%로 둘 사이에 있고 최대 동시 실행이 상한 17을 넘지 않는다(표 4).

Sukprasert의 시간 이동은 어느 리전 위에서 돌리느냐에 따라 두 행으로 갈린다. 홈 리전 위에서는 22.22%이지만, CAST(무릎점)가 정한 리전 위에서 시각만 옮기면 73.23%가 되어 CAST보다도 조금 더 줄인다. 그 차 51.01%포인트는 시간 이동이 아니라 공간 이동의 몫이다. 그러나 이때 필요 용량은 82건으로 기준 피크의 6.3배이고 위반은 37,236건이다. CAST가 상한까지 채워 둔 리전 위에서 Sukprasert의 시간 이동이 작업을 다시 같은 골로 몰기 때문이다.

Caspian은 용량 17을 자기 슬롯 단위의 하드 제약으로 두고 마감을 양보한다. 절감은 67.54%이고, 마감을 넘긴 작업이 5,582건(3.90%)이다. 탄소 가중치를 0.5~2로 바꿔도 67.34~67.97%로 거의 같다(원문 초록의 33%는 가중치 0.5의 값이다). 1시간 미만 작업을 실행 시간만큼의 분수로 세므로 자기 단위로는 상한을 지키지만, 시작 순간 단위로 다시 세면 1,538건이 상한을 넘고 필요 용량은 34로 기준 피크의 2.6배다. 같은 지연으로 맞춘 짝은 아니지만 평균 지연이 비슷한 CAST α=1(120.0 ms 대 115.7 ms)은 마감과 상한을 모두 지키며 28.11%포인트 더 줄이고(다만 CAST α=1은 지연을 목적에 넣지 않는다), CAST 무릎점도 4.39%포인트 더 줄인다. 원 논문은 경량 쿠버네티스 배포 도구(k3d)로 만든 클러스터 넷에 24시간 동안 작업 200건을 도착시켜 40시간을 평가했다[8].

순간 피크를 가린 99.9 백분위수로 다시 세어도 순서는 같다. CICS 9, Sukprasert 시간(홈) 19, CASPER 24, Caspian 28, Sukprasert 시간(본 연구 리전) 64, Sukprasert 공간+시간 107이다. CAST는 두 가중치 모두 17로, 순간 피크와 99.9 백분위수가 같다. 상한까지 채운 시간이 짧은 순간이 아니라는 뜻이다. Caspian을 빼면 모든 정책에서 마감을 넘긴 작업은 없었다. 가장 붐빈 리전으로 재면 CAST 무릎점보다 많이 줄이는 세 정책이 2.2~9.9배를 요구하지만, 리전별 최댓값을 모두 더한 설비 총량(표 7의 Total)은 CASPER(46), Caspian(86), CICS(88)가 CAST 무릎점(117)보다 작다. CASPER는 프랑스 한 곳에 29칸을 요구하는 대신 나머지 리전을 비운다. 리전 용량이

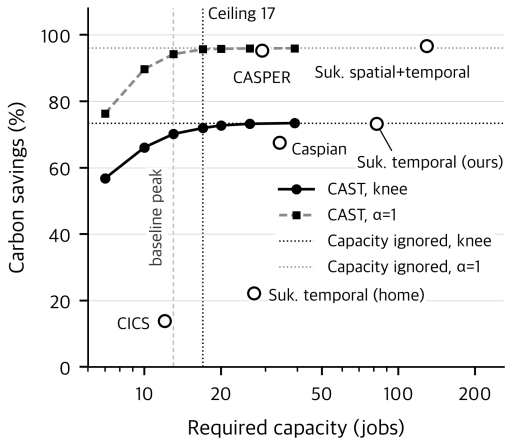


그림 11 선행 정책 재현 결과 비교(표 7). ○는 선행 정책, ●와 ■는 상한을 기준 피크의 0.5~3배로 바꾼 CAST의 무릎점과 α=1을 이은 것이고, 가로 점선은 같은 두 가중치에서 용량을 무시한 절감(검은 선 무릎점 73.39%, 회색 선 α=1 95.97%)이다. 회색 세로 파선은 기준 피크 13, 검은 세로 점선은 상한 17이며 가로축은 로그 눈금이다.

Fig. 11 Comparison of reproduced prior policies (Table 7). Hollow circles are prior policies; filled circles and squares connect CAST at the knee and at α=1 with the ceiling swept from 0.5× to 3× the baseline peak, and the horizontal dotted lines are the same two weights with capacity ignored (black: knee, 73.39%; gray: α=1, 95.97%). The vertical gray dashed line marks the baseline peak of 13 and the vertical black dotted line the ceiling of 17; the x axis is logarithmic.

데이터센터마다 정해져 있다는 본 연구의 가정에서는 한 리전의 크기가 맞는 자다.

4.9 계산 비용

CAST는 무릎점을 고르려고 매 슬롯 선형 계획을 열한 번 푼다. 1년 8,760 슬롯 동안 96,360번이며, 분수해가 나와 정수 계획으로 다시 푼 슬롯은 341개(3.9%)였다. 한 슬롯의 문제가 가장 컸을 때 변수는 7,848개였고, 정수 계획의 상대 최적성 허용오차는 HiGHS 기본값인 0.01%다. 무릎점 탐색을 포함한 슬롯당 풀이 시간은 평균 223 ms, 95 백분위수 380 ms, 최대 1,223 ms로 슬롯 길이 1시간에 비하면 무시할 만하며, 자료 적재와 채점까지 포함해 1년 전체를 4.1절의 기계 한 대에서 34.4분에 돌렸다. α 후보를 0.1 간격으로 둔 것은 계산과의 맞바꿈이다. 평가 구간 첫머리의 한 달(채점 작업 12,000건)에서 간격을 0.05로 좁혀 스물한 개를 풀면 절감은 66.19%에서 66.30%로 그 달 기준 배출의 0.11%포인트, 평균 지연은 1.1 ms 달라지지만 풀이 시간은 6.4분에서 12.4분으로 두 배가 된다.

α=1은 슬롯마다 한 번만 풀지만 정수 계획이 필요한 슬롯이 1,421개(16.2%)로 늘고 최대 변수가 9,488개라 1년에 14.4분이 걸린다. 분리형은 13.6분, 시간만은 7.7분, 공간만은 3.5분이다. 선형 계획 없이 작업마다 차례로 자리를 잡는 탐욕은 같은 무릎점 규칙으로 100.3분이 걸리지만, 순수 파이썬 구현이라 HiGHS를 쓰는 CAST와 속도를 견주지 않는다. 작업시간이 합성의 7.8배인 Azure 워크로드에서도 한 슬롯 문제의 최대 변수는 8,032개로 합성(7,848개)과 거의 같다. 변수 수가 작업 수가 아니라 대기열의 길이에 매이기 때문이다. 늘어난 것은 정수 계획으로 다시 푼 슬롯(712개, 8.1%)과 슬롯당 풀이 시간(평균 337 ms, 95 백분위수 616 ms, 최대 1,772 ms)이며, 1년 전체는 52.3분으로 합성의 1.5배다. α=1은 최대 변수 9,936개, 정수 계획 슬롯 1,588개(18.1%), 17.0분이다. 합성 워크로드의 도착을 두 배로 늘리면 최대 변수가 15,816개로 두 배가 되고, 1년 풀이 시간은 85.6분으로 2.5배다(슬롯당 평균 556 ms, 최대 4,298 ms).

5. 결론 및 향후 연구

본 논문은 탄소 인지 스케줄링에서 작업이 같은 깨끗한 시각과 리전으로 몰려 리전 용량을 넘기는 문제를 다루고, 확정 계획을 들고 있지 않고 매 슬롯 대기 작업 전부를 다시 배정하는 탄소 인지 스케줄링 시스템 CAST를 제안하였다. CAST는 매 슬롯 아직 시작하지 않은 작업 전부의 실행 리전과 시작 슬롯을 선형 계획으로 다시 정하고 리전 용량의 초과에 가장 큰 벌점을 둔다. 여덟 리전의 1년치 탄소집약도 위에서 평균 배출계수로 센 배출량을 무릎점에서 71.93%, 지연을 보지 않는 α=1에서 95.65% 줄였고, 작업 143,200건 전량을 마감 안에 처리하면서 상한을 넘긴 배정은 없었으며, 슬롯 안 도착을 미리 알지 않아도 그랬다. 위반이 없는 가장 가까운 두 설계와 견주면, 합성 워크로드에서 같은 고정 가중치로는 다시 풀지 않는 설계보다 1.19~1.64%포인트, 시작 때 빈 리전으로 보내는 설계보다 0.68%포인트 덜 배출했고, 무릎점 규칙으로는 같은 지연의 도착 고정형과 같은 만큼 배출하면서 작업을 31% 덜 기다리게 했다. Azure에서는 예약을 새로 도착하는 몫에 맞추면($\rho \le 6$) CAST가 앞서고, 넘어온 VM을 두 번 세어 커진 대표 설정(ρ=13)에서는 도착 고정형이 2.10%포인트 앞섰다. 남의 계획을 보지 않고 미래 시작을 확정하면 시작 때 확인이 없을 때 채점 작업의 9.0~17.0%가 넘쳤고, 확인해 빈 리전으로 보내면 넘치지 않았다. 예약 없이 탄소만 볼 때 용량을 지키는 비용은

합성 0.34%포인트, 평균 부하가 가장 깨끗한 리전의 상한을 넘는 Azure에서 27.37%포인트였다. 같은 지연 예산의 오프라인 하한과 견주면 계절이 다른 네 주에서 온라인 구조의 대가는 완전예지·고정 α로 0.13~0.64%포인트로 작았고, 무릎점의 격차(완전예지 1.87~2.77%포인트)는 대부분 무릎점 규칙에서, LSTM의 더 큰 격차(3.11~6.49%포인트)는 예측 오차에서 왔다. 가중치는 손으로 정하지 않는 기본값으로 슬롯마다 이상점에 가장 가까운 후보를 골랐지만, 목적식과 같은 자의 같은 지연 고정 가중치보다 합성 워크로드의 아홉 조건 가운데 여덟에서 1.77~2.64%포인트 더 배출했고, Azure에서는 작업당 평균 지연으로 잴 때만 앞섰다.

본 연구의 주된 발견은 용량을 무엇과 함께 확인하느냐가 결과를 가른다는 데 있다. 미래 시작을 확정한 계획들이 서로를 보지 못하면 같은 예측 최저 시각으로 모이고, 시작 때 확인하지 않는 한 넘치며, 시작 때 확인해 빈 리전으로 보내면 넘치지 않는다. 몰림 자체는 COAST[11]와 수요반응 연구[12]도 지적했다. 확정 계획을 모두 보거나 매 슬롯 다시 풀면 어떤 예측기에서도 위반이 없고, 미래 슬롯 행은 위반을 막는 것이 아니라 붐빌 슬롯을 미리 피해 절감을 0.12~0.72%포인트 더한다. 다만 공유 제약은 몰림을 지우지는 않는다. 몰린 작업은 골을 상한까지 채우는 데서 멈추고, 위반으로도 큰 손실로도 이어지지 않는다(4.5절).

평균 배출계수로 셀 때 상한이 걸리는 자리는 고르지 않다. 용량 행을 지운 CAST에서는 상한을 넘긴 시간의 93.3%가 공간 이동이 모이는 캘리포니아와 프랑스에서 나고, 총합이 113~114로 거의 같은 세 가지 리전별 상한 배치에서도 이 두 리전이 앞에 서며, 캘리포니아에서는 그 74.2%가 현지 9시부터 13시 전까지 몰린다. 시간 이동의 절감은 하루 절대 변동이 큰 다섯 리전이 92.8%를 내고, 두 목록에 함께 드는 리전은 캘리포니아와 일본이다(인도와 일본은 자료 산정 방식이 바뀐 것으로 보여 이 결과는 자료에 기댄다). 균일 상한과 kg 회계에서 이 결과의 대부분은 정의에서 따라 나오며, 상한 배치를 바꿔도 이 두 리전이 앞에 선다는 것이 측정이 보태는 몫이다.

이 결과가 실무에 주는 함의는 두 가지다. 첫째, 탄소 인지 스케줄링의 이득은 그 배치가 요구하는 용량과 함께 보고해야 비교할 수 있다. 1년치를 미리 아는 상한 분석인 Sukprasert 공간+시간의 96.63%는 리전 하나가 동시에 129건을 돌려야 성립하는 숫자로, 기준 피크 13의 9.9배다. CASPER의 95.30%도 기준 피크의 2.2배인 29건을 요구한다. 다만 리전별 최댓값을 모두 더한 설비 총량으로는 CASPER(46칸)가 CAST 무릎점(117칸)보다 작아, 이 비교는 리전마다 용량이 정해져 있다는 가정 위의 것이다.

둘째, 평균 배출계수로 절감을 절대량(kg)으로 셀 때 시간 이동을 도입할지는 하루 안의 상대 변동이 아니라 절대 변동, 곧 날마다 최댓값과 최솟값의 차로 판단해야 한다. 이는 회계 정의에서 따라 나오지만 상대 변동으로 판단하면 놓칠 수 있다. 프랑스는 상대 변동이 0.97로 큰 편이지만 절대 변동이 11.8 gCO_2/kWh로 가장 작아 0.3%만 낸다. 어느 축이 절감을 내는지는 워크로드가 정한다. 본 연구의 두 워크로드에서 짧고 미룰 수 있는 작업이 많으면 시간 이동과 결합 설계가 절감을 더했고, 길고 미룰 수 없는 작업이 많으면 공간 이동과 용량이 결과의 거의 전부였다.

한계는 다섯 가지다. 첫째, 탄소 회계가 Electricity Maps의 평균 배출계수 위에 있고, 받은 자료에 배출계수 기준과 추정값 여부가 표시되지 않으며, 프랑스 최솟값(2 gCO_2/kWh)으로 보아 직접 배출 기준으로 보인다. 화석 상류를 뺀 전과정 근사로 다시 채점하면 절감이 1.7~5.4%포인트 줄지만 순서는 같다(4.1절). 2025년 12월 31일 1~23시는 비어 있어 채운 값이다. 작업을 옮겨 실제로 바뀌는 한계 배출은 넣지 않았다. 프랑스로 보낸 밤 작업은 프랑스의 수출을 줄여 이웃 망의 화석 발전을 늘릴 수 있으므로 공간 이동의 절감은 한계 회계에서 작아질 수 있으며, 프랑스로 옮긴 몫만 독일 평균으로 센 시나리오에서는 무릎점이 38.99%로 줄고 α=1은 시간만과 비슷해지며 재풀이 이점도 사라지지만, 모든 강도를 화석 몫으로 센 대칭 대리에서는 같은 지연의 비교 순서가 남는다(4.1절). 설비 효율(PUE)과 그 시각별 변동, 유휴 전력도 넣지 않아, 절감률의 분모는 작업이 실행 중에 쓰는 전력의 배출이다. 추가 서버의 내재 배출도 세지 않았다. 리전마다 17칸이 이미 있다면 선행 정책이 더 요구하는 용량이 그만큼 배출을 더하지만, 필요한 만큼만 짓는다면 설비 총량이 큰 CAST 무릎점(117칸)이 불리하다. 미국 동부 리전은 지연을 Azure East US(버지니아)에서, 탄소를 뉴욕 전력망에서 가져온 혼합이고, 일본과 인도 자료는 데이터센터가 있는 하위 계통과 다를 수 있으며, 인도는 2024년 11월부터 하루 변동이 거의 두 배가 되어 학습 연도와 모양이 다르다. 리전 간 전송 에너지는 옮긴 작업마다 1 GB를 더해 보았을 때 목적지 강도로 세면 1%포인트 안, 출발지 강도로 세면 무릎점에서 5.92%포인트, α=1에서 8.00%포인트까지 줄었고 순서는 그대로였으나, 작업당 약 9 GB를 넘으면 CAST 무릎점이 시간만보다 못하다(4.3절). 둘째, 실제 배포가 아닌 시뮬레이션이어서 클러스터 관리 계층과의 연동 비용을 확인하지 못했고, 주 실험은 슬롯 안에 도착하는 작업을 슬롯 시작에

안다고 두어 최대 1시간을 미리 보며, 이를 없애면 위반은 그대로 0이고 절감은 0.56~1.62%포인트 준다. 탄소 쪽도 지금 슬롯의 첫 한 시간에 사후 확정값을 쓰고 발행 시각의 값을 예측 입력으로 써 한 시간을 미리 보며, 이를 없애도 위반은 0이고 LSTM 무릎점은 2.42%포인트, α=1은 0.17%포인트 낮아진다(4.2절). 셋째, 지연 등급과 합성 워크로드의 도착 시각은 모형값이고(도착 봉우리를 한낮으로 옮겨도 결론은 같다, 4.7절), 지연 항은 등급과 무관하게 실행 시간에 비례하는데, 이 형태는 설계 선택이라 등급별로 다른 비용을 두면 결과가 달라질 수 있다. Azure 워크로드도 도착과 실행 시간만 실측이고, 출발 리전은 균등하게 주었으며, VM 크기를 버려 VM 시간으로 가중한 값이다(코어 수에 비례한 전력으로 다시 채점해도 절감은 많아야 1.1%포인트 움직였으나, 용량은 여전히 VM 한 대를 한 자리로 센다). VM 수명을 미룰 수 있는 배치 작업으로 본 것과 실행 시간 d_j를 제출 때 안다고 둔 것도 가정이다. 실제 VM 수명은 미리 알 수 없으며 [29]는 그것을 예측하는 연구다. 이 지연 예산은 모든 작업에 공간 이동을 열어 두므로, 지연 상한을 100 ms로 두면 무릎점 절감이 32.98%로 꺾인다(4.7절).

넷째, 캘리포니아, 텍사스, 독일의 기상 입력이 예보가 아닌 실측값이다. 이 세 리전의 예측을 기상 입력이 없는 24시간 전 값으로 바꾸어도 절감은 무릎점에서 0.37%포인트, α=1에서 0.01%포인트 움직였다(4.2절). 다만 예측기 전체를 바꾸면 무릎점 절감이 64.44~71.93%로 움직여 무릎점 결과는 예측기에 매여 있으며, 그 차이의 대부분은 무릎점 규칙이 예측 곡선의 모양에 따라 다른 α를 고르는 데서 오고, 같은 지연의 고정 α 하나는 72.6~74.0%로 덜 흔들린다(4.5절). 무릎점은 선형 계획 완화의 좌표로 고르고 분수해가 난 슬롯만 그 α에서 정수 계획으로 다시 풀므로, 그 슬롯들에서는 정수해의 좌표로 고른 무릎점과 다를 수 있다. 다섯째, 탄소 자료가 2025년 한 해이고 예측 지평이 24시간이어서, 하루를 넘는 작업의 리전도 앞 24시간 예측으로 정하며, 평균 창이 지평 끝에서 잘려 Azure의 긴 작업은 지평 안의 평균으로 판단된다. 같은 α에서 결합형과 분리형을 견준 비교는 주 실험의 시드 한 벌에서만 했고, 최적성 격차는 네 주와 같은 시작 후보 격자, 상한 17에서만 쟀고 24시간 지평의 대가는 재지 못했으며, 같은 지연 짝의 맞춤 오차(Azure 짝의 지연 차 0.58~0.70 ms만큼)는 부트스트랩 구간에 들어 있지 않으며, 최악의 경우에 대한 경쟁비 보장은 없다. 위반 0은 공간 자유도와 집계 여유(최대 동시 실행 합이 총 상한의 합성 46/136, Azure 153/344)에서 따라 나오며, 이용률이 가장 높은 조건(Azure 43.2%)까지만 확인했고, 그 위의 과부하에서는 식 (11)의 초과 변수가 실제로 양수가 될 수 있다. 선행 정책은 원문 수식을 따르되 바꾼 점이 있고(4.8절), 원 논문의 자료가 아니어서 원 수치의 재현은 확인하지 못했다.

향후 과제는 실제 예보 API 연동, 한계 배출계수를 쓰는 회계, 24시간을 넘는 예측 지평[33], 그리고 리전 상한을 수요와 설비에서 추정하는 방법이다. 슬롯마다 다른 분모로 정규화한 탓에 슬롯별 무릎점은 슬롯마다 다른 교환비를 쓰므로, 교환비를 하나로 맞추는 방법(목표 지연 예산에 맞춘 라그랑주 승수의 갱신)도 향후 연구로 남긴다. 마지막은 Zanotto 등이 남긴 과제이기도 하며[9], 본 연구가 상한을 실험 변수로 두고 바꿔 가며 본 결과가 그 출발점이 될 수 있다. 용량을 지키면서 얻은 절감이 실제 클러스터에서도 유지되는지 검증할 계획이다.

참고문헌

[1] International Energy Agency, Energy and AI, World Energy Outlook Special Report, IEA, Paris, 2025.

[2] European Parliament and Council, Directive (EU) 2023/1791 of the European Parliament and of the Council of 13 September 2023 on Energy Efficiency (recast), Official Journal of the European Union, 2023.

[3] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, “Carbon-Aware Computing for Datacenters,” IEEE Transactions on Power Systems, Vol. 38, No. 2, pp. 1270-1280, Mar. 2023.

[4] A. Souza, S. Jasoria, B. Chakrabarty, D. Irwin, P. Shenoy, A. Bridgwater, A. Lundberg, F. Skogh, and A. Ali-Eldin, “CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services,” Proc. of the 14th Int. Green and Sustainable Computing Conf. (IGSC), pp. 67-73, Oct. 2023.

[5] D. Maji, B. Pfaff, V. P R, R. Sreenivasan, V. Firoiu, S. Iyer, C. Josephson, Z. Pan, and R. K. Sitaraman, “Bringing Carbon Awareness to Multi-Cloud Application Delivery,” Proc. of the 2nd Workshop on Sustainable Computer Systems (HotCarbon), pp. 1-6, Jul. 2023.

[6] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, “On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud,” Proc. of the 19th European Conf. on Computer Systems (EuroSys), pp. 924-941, Apr. 2024.

[7] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, “CarbonFlex: Enabling Carbon-aware Provisioning and Scheduling for Cloud Clusters,” arXiv:2505.18357, 2025.

[8] T. Bahreini, A. N. Tantawi, and O. Tardieu, “Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments,” Proc. of the 32nd IEEE Int. Symp. on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), pp. 1-8, Oct. 2024.

[9] M. Zanotto, L. Vicentini, R. Vreto, F. Lumpp, D. Braga, and S. Fiore, “A User-centric Kubernetes-based Architecture for Green Cloud Computing,” arXiv:2509.13325, 2025.

[10] Green Software Foundation, Carbon Aware SDK, v1.8.0 [Online]. Available: https://github.com/Green-Software-Foundation/carbon-aware-sdk (downloaded 2026, Oct. 18)

[11] W. Feng, A. Dasgupta, Z. D. Tekler, S. Ahuja, J. Zheng, and X. Jiang, "COAST: Congestion-Aware Start-Time Recommendations for Carbon-Aware HPC Jobs," arXiv:2609.05443, 2026.

[12] C. Han, D. Witthaut, L. Rydin Gorjão, and P. C. Böttcher, "Collective Effects and Synchronization of Demand in Real-Time Demand Response," Journal of Physics: Complexity, Vol. 3, No. 2, 025002, 2022.

[13] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud," Proc. of the 22nd Int. Middleware Conf. (Middleware), pp. 260-272, Dec. 2021.

[14] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, "CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency," Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 7, No. 3, Article 57, Dec. 2023.

[15] W. A. Hanafy, Q. Liang, N. Bashir, A. Souza, D. Irwin, and P. Shenoy, "Going Green for Less Green: Optimizing the Cost of Reducing Cloud Carbon Emissions," Proc. of the 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3, pp. 479-496, Apr. 2024.

[16] K. Xu, D. Sun, H. Tian, J. Zhang, and K. Chen, "GREEN: Carbon-efficient Resource Scheduling for Machine Learning Clusters," Proc. of the 22nd USENIX Symp. on Networked Systems Design and Implementation (NSDI), pp. 999-1014, Apr. 2025.

[17] G. Attenni, Y. Moawad, N. Bartolini, and L. Thamsen, "Spatio-Temporal Shifting to Reduce Carbon, Water, and Land-Use Footprints of Cloud Workloads," arXiv:2512.08725, 2025.

[18] A. Lechowicz, N. Christianson, B. Sun, N. Bashir, M. Hajiesmaili, A. Wierman, and P. Shenoy, "Learning-Augmented Competitive Algorithms for Spatiotemporal Online Allocation with Deadline Constraints," Proc. of the ACM on Measurement and Analysis of Computing Systems (POMACS), Vol. 9, No. 1, Article 8, pp. 1-49, Mar. 2025.

[19] S. Hall, F. Micheli, G. Belgioioso, A. Radovanović, and F. Dörfler, "Carbon-Aware Computing for Data Centers with Probabilistic Performance Guarantees," arXiv:2410.21510, 2024.

[20] D. Maji, R. K. Sitaraman, and P. Shenoy, "DACF: Day-Ahead Carbon Intensity Forecasting of Power Grids using Machine Learning," Proc. of the 13th ACM Int. Conf. on Future Energy Systems (e-Energy), pp. 188-192, Jun.-Jul. 2022.

[21] J. Edmonds, "Submodular Functions, Matroids, and Certain Polyhedra," in Combinatorial Optimization — Eureka, You Shrink!, LNCS 2570, pp. 11-26, Springer, 2003.

[22] A. Schrijver, Theory of Linear and Integer Programming, Wiley, Chichester, 1986.

[23] A. J. Hoffman and J. B. Kruskal, "Integral Boundary Points of Convex Polyhedra," in H. W. Kuhn and A. W. Tucker (Eds.), Linear Inequalities and Related Systems, Annals of Mathematics Studies 38, Princeton University Press, pp. 223-246, 1956.

[24] J. Branke, K. Deb, H. Dierolf, and M. Osswald, "Finding Knees in Multi-objective Optimization," Proc. of the 8th Int. Conf. on Parallel Problem Solving from Nature (PPSN VIII), LNCS 3242, pp. 722-731, Sep. 2004.

[25] M. Zeleny, "Compromise Programming," in J. L. Cochrane and M. Zeleny (Eds.), Multiple Criteria Decision Making, University of South Carolina Press, Columbia, pp. 262-301, 1973.

[26] Electricity Maps. API Documentation [Online]. Available: https://docs.electricitymaps.com (downloaded 2026, Oct. 12)

[27] Electricity Maps. Methodology [Online]. Available: https://www.electricitymaps.com/data/methodology (downloaded 2026, Sep. 22)

[28] Microsoft. Azure Network Round-Trip Latency Statistics [Online]. Available: https://learn.microsoft.com/en-us/azure/networking/azure-network-latency (downloaded 2026, Sep. 22)

[29] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms," Proc. of the 26th ACM Symp. on Operating Systems Principles (SOSP), pp. 153-167, Oct. 2017.

[30] Microsoft Azure, Azure Public Dataset V2, 2019 [Online]. Available: https://github.com/Azure/AzurePublicDataset (downloaded 2026, Oct. 17)

[31] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, "Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis," Proc. of the 3rd ACM Symp. on Cloud Computing (SoCC), Article 7, Oct. 2012.

[32] J. Aslan, K. Mayers, J. G. Koomey, and C. France, "Electricity Intensity of Internet Data Transmission: Untangling the Estimates," Journal of Industrial Ecology, Vol. 22, No. 4, pp. 785-798, 2018.

[33] D. Maji, P. Shenoy, and R. K. Sitaraman, "CarbonCast: Multi-Day Forecasting of Grid Carbon Intensity," Proc. of the 9th ACM Int. Conf. on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys), pp. 198-207, Nov. 2022.